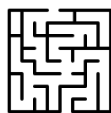
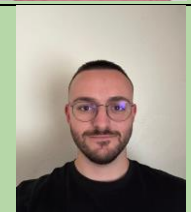
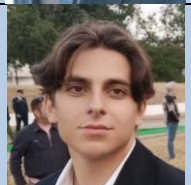


PI de Sistemas de Informação Distribuídos (24/25)

Licenciaturas em Engenharia Informática e Informática e Gestão de Empresas



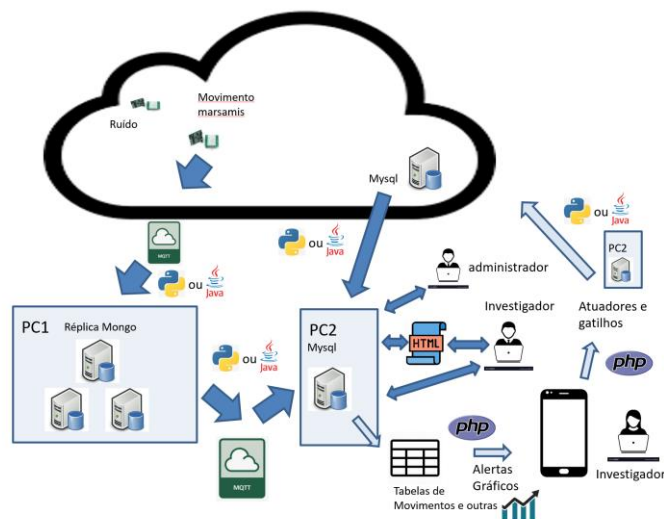
Grupo 24		Grupo 15	
111006 Alexandre Plotean apnel1@iscte-iul.pt		110993 Andre Soares assss3@iscte-ul.pt	
68710 Catarina Silva Correia cscaa11@iscte-iul.pt		110005 Bruno Silva bfbps@iscte-ul.pt	
104671 Mário Humberto Simões Veloso Vieira mhsvv@iscte-iul.pt		106483 Diogo Vida-Larga dmvde@iscte-ul.pt	
106771 Misaél Agostinho Armando maaoo@iscte-iul.pt		106484 Diogo Perez dspzo@iscte-ul.pt	
112029 Nuno Faísca Coelho nfcoa2@iscte-iul.pt		111171 Rafael Nicolau rcgnu1@iscte-ul.pt	
93352 Ricardo Miguel Pombo Gonçalves Maruta rmpgm@iscte-iul.pt		106584 Pedro Torrado petsa@iscte-ul.pt	

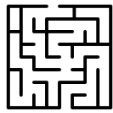


Instruções

Estas instruções são de cumprimento obrigatório. Relatórios que não cumpram as indicações serão penalizados na nota final.

- Podem (e em várias situações será necessário) ser adicionadas novas páginas ao relatório, mas não podem ser removidas páginas. Se uma secção não for relevante, fica em branco, não pode ser removida;
- A paginação tem de ser sequencial e não ter falhas;
- O índice tem de estar atualizado.
- O grupo que inicia o documento (coluna à esquerda na folha de rosto) preenche apenas a parte inicial (até ao fim da secção secção 1). Este documento word vai ser colocado no moodle para que o outro grupo (à direita da folha de rosto) possa descarregar e continuar a preenche-lo (secção 2)





Índice

1. Especificação	5
1.1. Da Nuvem para o Mongo	5
1.2. Descrição Geral do Procedimento de Mongo Para Mysql	8
1.4. Tratamento de dados anómalos (valores de sensores errados)	13
1.5. Tratamento de outliers de ruído	15
1.6. Tratamento de Alertas de ruído	16
1.7. Tratamento de número de marsamis numa sala (obter pontuação)	17
1.8. Especificação de Store Procedures SQL de apoio à migração e tratamento de dados	19
1.9. Especificação de Triggers de apoio à migração e tratamento de dados	20
1.10. Modelo Relacional	21
1.11. Utilizadores Base de Dados Mysql	22
1.12. Procedimentos Manutenção da Aplicação	24
1.13. Eventos de suporte à aplicação (caso existam)	25
1.14. Consulta por HTML/PHP	26
2. Implementação	29
2.1. Migração Dos Dados	29
2.1.1. Criação da BD na MongoDB	29
2.1.2. Descrição Geral do Procedimento de Mongo Para Mysql	31
2.1.2.1. Da Cloud para o Mongo	31
2.1.2.2. De Mongo para MQTT	31
2.1.2.3. De MQTT para MySQL	33
2.2. Tratamento dos Dados	37
2.2.1. Tratamento de dados anómalos (valores de sensores errados)	37
2.2.2. Tratamento de outliers de ruído	39
2.2.3. Tratamento de Alertas de Som	40
2.2.4. Tratamento de número de marsamis numa sala (obter pontuação)	41
2.2.5. Implementação de Stored Procedures SQL de apoio à migração e tratamento de dados	43
2.2.6. Implementação de Triggers	44
2.3. Base de Dados	45



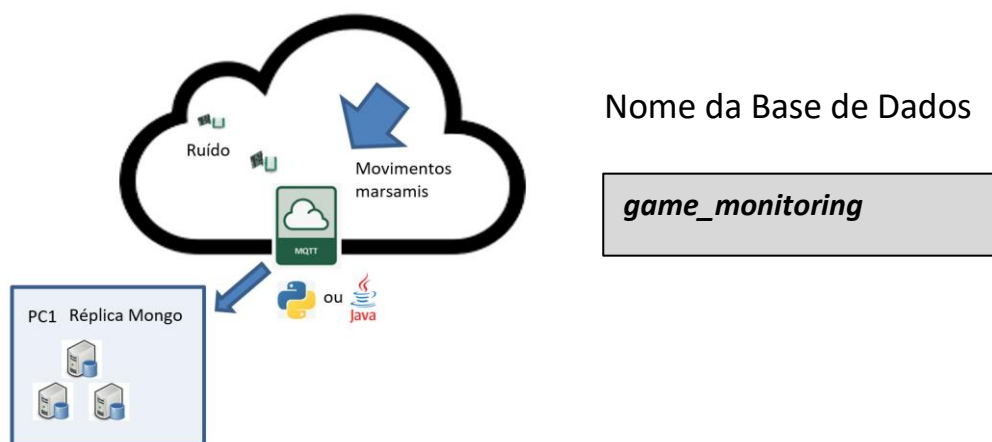
2.3.1.	Modelo Relacional	45
2.3.2.	Utilizadores Base de Dados MySQL	48
2.4.	Aplicação	50
2.4.1.	Procedimentos de Manutenção da Aplicação	50
2.4.2.	Eventos de suporte à aplicação (caso existam)	52
2.5.	Anexos de PrintScreen	53
2.5.1.	PrintScreen dos formulários HTML/PHP implementados	53
2.5.2.	PrintScreen dos gráficos Android com dados	55
2.6.	Anexos Códigos MySQL	57
2.6.1.	Código Exportado do phpMyAdmin	57
2.6.2.	Códigos dos Triggers Implementados	63
2.6.3.	Códigos dos SPs Implementados	68



1. Especificação

Esta secção é onde o grupo que inicia o documento (coluna à esquerda na folha de rosto) coloca a especificação do que pretende implementar. Mais tarde pode implementar de outra maneira, mas aqui vão as primeiras ideias que serão avaliadas na primeira oral e que vão ser entregues a outro grupo para que analisem e vejam se aproveitam as vossas ideias.

1.1. Da Nuvem para o Mongo



Nome Coleção	O que armazena?
raw_messages	Mensagens brutas do MQTT antes de qualquer tratamento.
move_messages	Movimentos validados as durante a sessões ativas.
sound_messages	Medições de som durante a sessão.
failed_messages	Mensagens com falhas de processamento de todas as sessões.

Para cada coleção exemplifica um documento

Coleção : **raw_messages**

{

"_id": "67de8a1bf2d34e89ab12c5d7",

"received_at": "2025-03-22T08:00:45.789+00:00",

"topic": "pisid_mazemov_33",



```
"raw_payload": "{ \"player\": 33, \"action\": \"move\", \"marsami\": 15,
\"room_origin\": 3, \"room_destiny\": 4, \"status\": 1 }",

"hash": "d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6",

"message_id": 123

}
```

Coleção : **move messages**

```
{

"player": 33,

"timestamp": "2025-03-22T08:01:22.456+00:00",

"marsami": 15,

"room_origin": 3,

"room_destiny": 4,

"status": 1,

"message_hash": "a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x",

"processed": false

}
```

Coleção : **sound messages**

```
{

"player_id": 33,

"hour": "2025-03-22T08:02:30.987+00:00",

"sound": 65.4321987654321,

"message_hash": "z9y8x7w6v5u4t3s2r1q0p9o8n7m6l5k4j3i2h1g0f9e8d7c6b5a",

"processed": false

}
```



Coleção : **failed messages**

{

"_id": "67de9b2cf3e45f90bc23d6e8",

"received_at": "2025-03-22T08:03:15.321+00:00",

"topic": "pisid_mazemov_33",

"raw_payload": "{

\"player\": 33,

\"marsami\": 10,

\"room_origin\": 1,

\"room_destiny\": 2,

\"status\": \"invalido\"

}\",

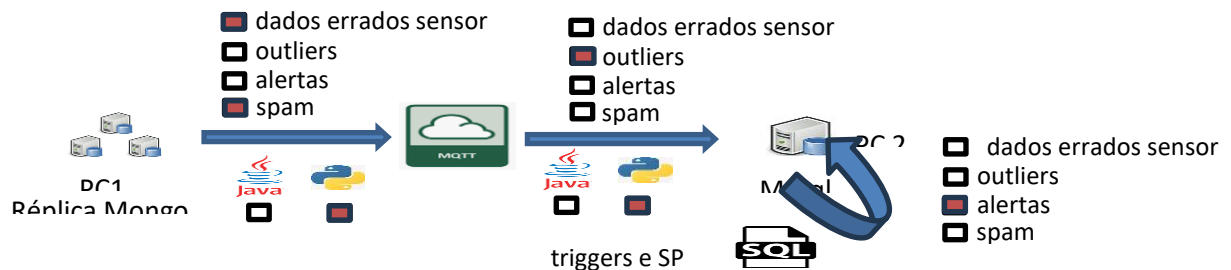
"error": "Valor de status inválido"

}



1.2. Descrição Geral do Procedimento de Mongo Para Mysql

A passagem do Mongo para Mysql tem duas fases: a) enviar de Mongo para MQTT e b) receber de MQTT para Mysql. No diagrama deve ser indicado (nas check box) em qual das fases são programadas (em java e/ou python) as tarefas enumeradas (podem ser na fase a) b) ou ambas).



- Com que “periodicidade” (segundos) o programa vai buscar ao mongo: do mongo para o script python streaming (0 segundos) e script python para o MQTT segue em batches de 0,25 segundos
- Como garantem que não enviam duas vezes o mesmo documento do Mongo para o Mysql / MQTT (com base em datas ou booleano ou etc.):

De modo a garantirmos que apenas é enviado uma vez o mesmo documento tivemos a seguinte abordagem: dividir o processo de enviar do mongo para o MQTT em duas partes, nomeadamente receber os novos dados do mongo e enviá-los para o MQTT. Referindo-nos agora ao mongo, nós fazemos a leitura através de uma feature providenciada pelo mongo, chamada “Streaming” o que nos permite ter “notificações” em tempo real de novos documentos adicionados ao mongo, com a segurança de não se perder nenhum e sem receber repetidos. Após o recebimento dos novos documentos no python vamos



proceder ao envio para o PC2 (MySQL), para isso vamos usar uma MQTT através de um broker. Para garantirmos que os dados são recebidos pelo cliente apenas uma vez e que apenas serão eliminados do MQTT quando o cliente já os tiver processado, usamos uma MQTT com QOS de valor 2.

- Pensam usar threads do Mongo para MQTT?: 4 e do MQTT para Mysql 2?

- Quantas threads e/ou quantos main em cada um dos passos?

No processo do Mongo para MQTT usamos 4 threads. Duas para o movimento dos marsamis e outras duas para o som. Note que, para cada um destes dois tipos (de threads) existem duas threads, uma para receber os dados vindos do mongo e outro para periodicamente enviar os dados para o MQTT, respetivamente.

Relativamente ao processo do MQTT para o MySQL, usamos apenas duas threads uma para receber os movimentos e outra para receber os dados do som. De cada vez que uma destas threads recebe um batch de informação e faz o tratamento da informação e depois envia para o MySQL.

- No transporte MQTT que QOS vão utilizar? Porquê?

Iremos usar o QOS de valor 2, de forma a garantir uma maior segurança no processamento dos documentos. Garantindo que os documentos chegam ao cliente, apenas uma vez e que principalmente apenas iram removidos da MQTT quando o cliente acabar de os processar.



Aqui podem desenvolver informação que considerem relevante relativo ao processo de migração, aspectos que não esteja refletido nas secções seguintes.

Neste ponto, vai ser explicado como através do script que passa as informações do Mqtt para o MySQL se organiza a tabela Mensagens no MySQL.

Tabela Mensagem: (guarda informação de todas as mensagem que forem passadas)

hora: timestamp(informação passada através do Mongo)

sala:

Exemplos:

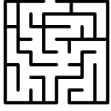
- 0 se a informação vier do som*
- x, se informação vier do movimento, sendo x a sala de origem.*

Sensor: dois valores possíveis, 1 e 2, de acordo com o sensor de movimento, e o sensor de som.

leitura:

Exemplos:

- x , se a informação vier do som, em que x é o nivel atual de som no jogo*
- y.00 , se a informação vier do movimento, e y a sala destino*



tipoAlerta: Apenas é escrito caso hajam alertas relevantes para guardar, senão o campo fica vazio

Exemplo:

- se o som passar de 80% do valor limite o script cria um texto "alerta de som"

- outras mais informações que sejam relevantes para o jogo.

mensagem: visto ser um varchar(100), o script cria um texto de acordo com a informação recebida.

Exemplo para movimento:

- se room_origin = 1 e room_destiny = 2, "marsami x, movimentou se da Sala y para a Sala z"

- se room_origin = 0 e room_destiny = 0, "marsami x, preso na sala y"

- se status = 2, "marsami x cansado"

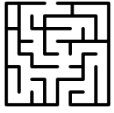
Exemplo para som:

- se sound = x, "nivel de som do jogo é x"

- se sound = x e x >= (80% limite maximo do som), "o nivel de som esta a 80% do limite"

horaEscrita:

- hora atual de escrita da informação na tabela, atraves de um CURRENT_TIMESTAMP





1.4. Tratamento de dados anómalos (valores de sensores errados)

Aqui devem explicar o que fazer caso se detectem valores "errados" (datas impossíveis, caracteres estranhos, etc..). Se recorrerem a triggers ou SP então indicam em secção mais adiante.

Para mensagens com dados anómalos como por exemplo mensagens mal formatadas, com valores de som negativos ou ainda, com datas que não estejam/sejam corretas, as validações serão bastante diretas.

Uma das validações será ver se existem campos em falta nas mensagens recebidas (tendo em conta o tipo de mensagem: tipo movimento e tipo som).

Uma segunda validação, apenas para as mensagens do tipo som, será validar se o valor do som é maior que 0 (valor positivo).

Uma terceira validação, para mensagens com datas erradas será validar se a data recebida (timestamp/data quando a mensagem é gerada na cloud) e a data atual (timestamp/data no momento da receção da mensagem no mongo) são próximas (i.e a diferença entre elas é inferior ao um pré-determinado "threshold" (de 2 a 3 segundos). Esta diferença permite acomodar ligeiros delays. Caso o delay seja superior ao "threshold" definido para o efeito, estas mensagens são colocadas na tabela de mensagens falhadas.

Uma quarta validação, relativa ao ID do jogador garante que, sempre que um jogo novo começa, no mongo é recebida uma mensagem a avisar que o jogo começa e a indicar o id do jogador. E, o ID do



jogador é guardado no script do MQTT para o mongo, o que vai fazer com que este script consiga validar que todas as mensagens recebidas daquele jogo têm o mesmo ID de jogador.

Por fim, foi definida uma quinta validação, responsável por validar se o corredor usado pelo movimento do marsami é válido. No início de cada jogo é feito uma consulta à bd na nuvem. Os dados (tabelas corredor e setupmaze) consultados ficarão armazenados num script python.

Notando que, do mongo para o python a informação é recebida via streaming, permitindo uma atualização dos dados em real-time.

Antes que possa ser adicionada ao mongo, uma nova mensagem (recebida via streaming), esta passa por um processo de filtragem. Este processo de filtragem possibilita que apenas dados de movimentos cujos corredores são válidos (o que implica a flag ficar processed) são recebidos pelo python, todos os restantes são assim ignorados pelo python (este quinto processo é feito pelo PC2).

Consideramos relevante mencionar que, todas as mensagens não válidas, serão enviadas para a tabela de mensagens falhadas, para futuras análises sobre dados errados (como por exemplo, percentagem de mensagens erradas, quais os tipos de mensagens com maior probabilidade de mensagens erradas, etc ...).



1.5. Tratamento de outliers de ruído

Aqui devem explicar o que fazer caso se detectem "outliers" (valores fisicamente possíveis mas irrealistas, como por exemplo variações muito bruscas do ruído apenas num segundo). Se recorrerem a triggers ou SP então indicam em secção mais adiante.

Tal como no ponto anterior (1.4), onde existia uma quinta validação responsável por validar os movimentos de corredor, neste ponto será feito exatamente o mesmo processo, mas desta vez para a validação de outliers de ruído (dados da tabela setupmaze). Isto significa que, no início de cada jogo é feito uma consulta à bd na nuvem e os dados (tabelas corredor e setupmaze) consultados que, ficarão armazenados num script python.

Esta validação garante que, o noise atual do jogo não varia mais do que o valor do "noisevartoleration". Caso a variação entre a mensagem de som atual e o valor do som da mensagem anterior supere o "noisevartoleration" é considerada com outlier e fica com a flag processed false. No fim do jogo, estas mensagens flagged como não processadas serão colocadas na tabela de mensagens falhadas.



1.6. Tratamento de Alertas de ruído

Aqui devem explicar o que fazer caso se detectem situações alarmantes relativos ao som. Se recorrerem a triggers ou SP então indicam em secção mais adiante. Qu situações despoletam os alertas, onde e como são armazenados. Explicar se existem mecanismos para evitar "spam" (demasiadas mensagens). É aconselhável recorrer a esquemas gráficos para explicar o mecanismo.

Os alertas de ruído vão ser feitos diretamente através do script que vem do MQTT para o MySQL, porque pelo script podemos ter acesso direto ao limite de som. Caso o limite ultrapasse os 80%, vamos escrever diretamente na tabela Mensagens, no atributo TipoAlerta: "Alerta som", e no atributo Mensagem: "o nível de som esta a 80% do limite".



1.7. Tratamento de número de marsamis numa sala (obter pontuação)

Aqui devem explicar como funciona o mecanismo que detecta que o número de marsamis odd é (ou vai ser) igual ao número de marsamis even. Como detecta, e o que desencadeia.

O grupo preparou duas alternativas para tratar do número de Marsamis numa sala, aquando da implementação escolheremos qual a melhor solução.

(Solução número 1)

- Na situação em que numa sala temos o número de Marsamis Odd menor que Even, ou Even menor que Odd, e a diferença entre eles seja igual a 1, vamos ter no script python uma função que vai saber se o próximo movimento para essa mesma sala vem um Marsami Odd ou Even. Caso o próximo Marsami a chegar a essa sala seja o tipo de Marsami desejado (para "casar" o mesmo número de Odds e Evens), vamos chamar o atuador CloseDoors de entrada e de saída para essa mesma sala (para qual o Marsami se vai movimentar).

De seguida, quando recebermos a próxima informação de movimento de Marsami, vamos ter um SP que tem um tempo de espera de 0.6segundos (o que equivalente a 3 mensagens) e após deste período de espera verifica as tabelas das salas (na bd MySQL do PC2).

Se aparecem nestas tabelas movimentos em marsimis que deviam ter ficado fechados na sala (Marsamis estes, que não deveriam ter tido novos movimentos na tabela) significa que as portas de entrada e saída não foram fechadas a tempo e o Marsami se movimentou se para outras salas. Caso isto aconteça, abrimos as portas de forma que todos os



Marsamis dessa sala se possam voltar a movimentar.

Caso as portas de entrada e saída tenham sido fechadas a tempo e como tal, não apareceram nestas tabelas movimentos em Marsamis que deviam ter ficado fechados na sala, chamamos o atuador Score, de forma a obter a pontuação Máxima dessa sala e em seguida, voltamos a abrir as portas, e a permitir a movimentação desses Marsamis.

(Solução número 2)

- Vamos ter um algoritmo em python, que logo no início do jogo, vai fechar todas as salas e verificar quantas Marsamis temos em cada sala. Caso em alguma sala tenhamos o mesmo número de Odds e Evens, chamamos o atuador Score para essas mesmas salas, de forma a obter o número máximo de score para essa sala.

Em seguida, o algoritmo através de grafos (em que cada node é uma sala), vamos obter o caminho mais rápido, de forma que seja o próprio algoritmo a controlar todos os movimentos dos Marsamis, e obter o caminho mais rápido, para que consigamos obter o máximo de pontuação no menor tempo possível.



1.8. *Especificação de Store Procedures SQL de apoio à migração e tratamento de dados*

Nas secções anteriores foram descritos mecanismos que podem ou não necessitar de recorrer a Store Procedures. É nesta tabela que eles deverão ser listados. Na descrição apenas colocar informação que não seja óbvia.

Nome SP	Argumentos	Muito breve descrição
openDoor	porta_origem, porta_destino	Abre o corredor entre uma porta de origem e destino
startGame		Muda o status do jogo para status "jogo_iniciado"
openAllDoors		Abre todos os corredores entre as portas de origens e destinos
closeDoor	porta_origem, porta_destino	Fecha o corredor entre uma porta de origem e destino
closeAllDoors		Fecha todos os corredores entre as portas de origens e destinos
getPoints	sala_id	Executa a acção do cliente ganhar pontos numa sala, vai ver quantos marsamis estão numa dada sala e consoante o valor vai adicionar 1 ou perde .5 pontos



1.9. Especificação de Triggers de apoio à migração e tratamento de dados

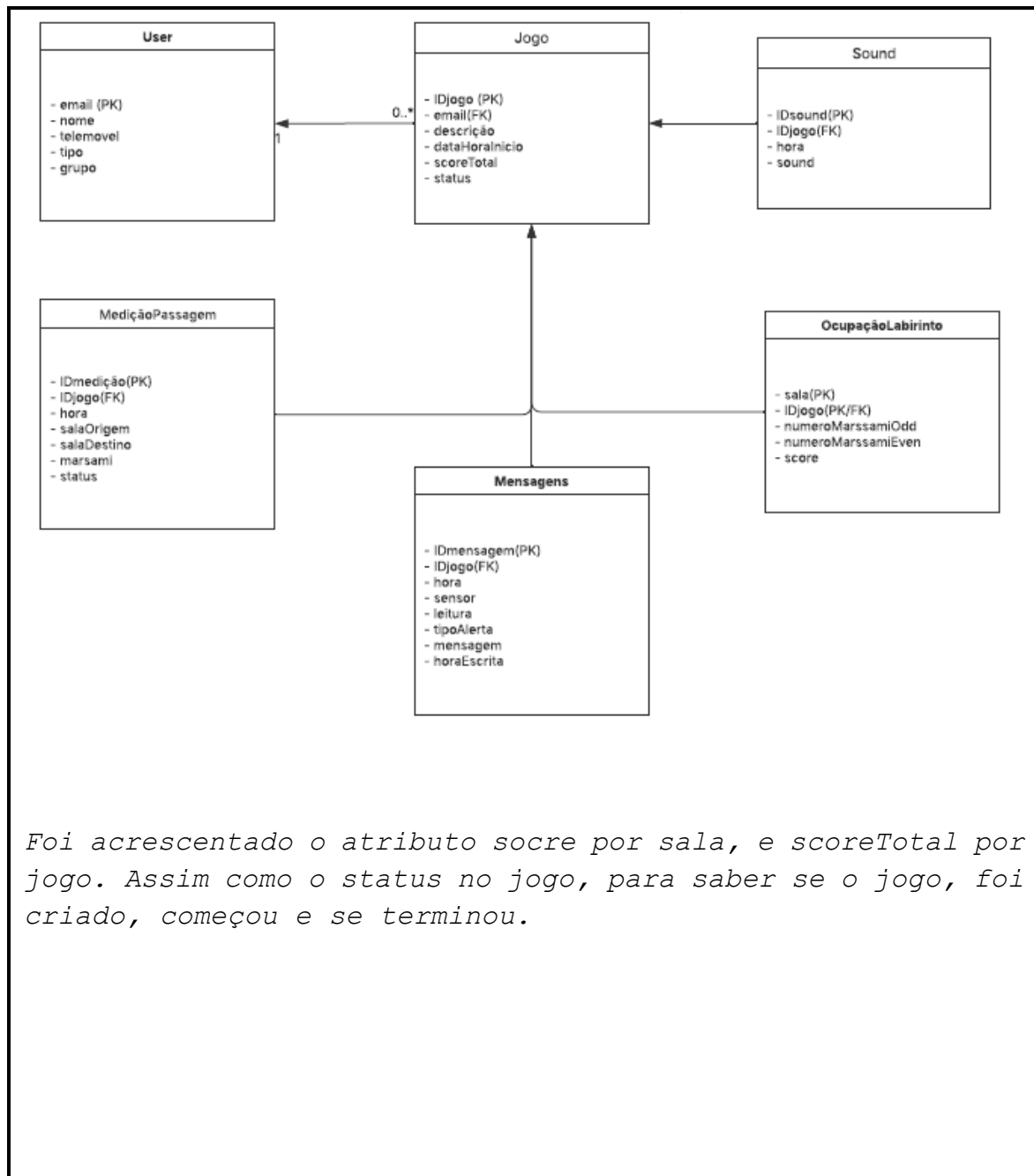
Nas secções anteriores foram descritos mecanismos que podem ou não necessitar de recorrer a Triggers. É nesta tabela que eles deverão ser listados. Nas Notas apenas colocar informação que não seja óbvia.

Nome Trigger	Tabela	Tipo de Operação (I,U,D)	Evento (After, Before)	Notas
updateNumberMarsamis	MediçãoPassagem	I	AFTER	Atualiza o número de marsamis odd/even na tabela OcupaçãoLabirinto
insertPassage	Mensagens	I	AFTER	Insere uma nova linha na tabela MediçãoPassagem
insertSound	Mensagens	I	AFTER	Insere uma nova linha na tabela Sound



1.10. Modelo Relacional

Diagrama relacional completo. Alterações à base de dados original terão de ser justificadas aqui. Caso seja pertinente podem ser adicionadas comentários a justificar opções pouco óbvias.





1.11. Utilizadores Base de Dados Mysql

Nesta secção deverá ser explicado de que forma deverá ser feita a manutenção de utilizadores Mysql. Nomeadamente deverá ser indicado, para cada tipo de utilizador, que privilégios ele tem sobre que tabelas e Stored Procedures (todos os SP usados na Aplicação

Tabela	Tipo de Utilizador		
	Administrador	Jogador	Tester
Jogo	-	U/I/L	L
Utilizador	U/D/I/L	U	U
Sound	-	I/L	L
MediçõesPassagens	-	I/L	L
Mensagens	-	I/L	L
OcupaçãoLabirinto	-	I/L/U	L
openDoor (SP)	-	X	X
startGame (SP)	-	X	X
openAllDoors (SP)	-	X	X
closeDoor (SP)	-	X	X
closeAllDoors (SP)	-	X	X
getPoints (SP)	-	X	X
endGame (SP)	-	X	-
Criar_utilizador (SP)	X	-	-
Remover_utilizador (SP)	X	-	-
	X	X	X



Alterar_utilizador (SP)			
Criar_jogo (SP)	-	X	-

Em que U=Update, I Insert, D- Delete, L=Leitura, X=Executar SP e - sem permissões.



1.12. *Procedimentos Manutenção da Aplicação*

Nesta secção deverão ser listados os SP para a manutenção de utilizadores e jogos (apenas os obrigatórios)

Nome SP	Argumentos	Muito breve descrição
Cria_rut ilizador	Email, Nome, Telemóvel, Tipo, Grupo, Pass	Cria o utilizador na também é adicionado da base de dados do MySQL com uma password
Remover_u tilizador	Email	Remove da base de dados do MySQL, se não estiver a jogar
Alterar_u tilizador	Email, Pass, Nome, Telemovel, Tipo, Grupo	Altera todas as informações sobre do utilizador, se a password estiver vazia não é alterada
Criar_jog o	Descrição, EmailJogador, DataHorainicio	Cria um jogo, com o status 0 ("Jogo_criado")
Alterar_j ogo	ID_Jogo, Descrição	Altera a informação de um dato jogo se este não tiver começado



1.13. Eventos de suporte à aplicação (caso existam)

Deverão ser indicados todos os eventos relevantes para o processo de migração, eventos do Windows e do Mysql. Por eventos entende-se as tarefas do Windows ou eventos do Mysql

Nome Evento	Local Execução (Mysql/Windows)	Muito breve descrição
auditarMensagensMongo	Windows	Serve para quando um jogo termina fazer um loop por todas as mensagens do mongo DB e verificar se foram marcadas como não processadas, caso afirmativo move as para a tabela de mensagens falhadas estas para futuras estatísticas.



1.14. Consulta por HTML/PHP

Desenhar o layout dos formulários pretendidos, se relevante colocar texto a explicar a funcionalidade pretendida. Formulários:

Fazer login

Criar (ou selecionar de uma lista de jogos um para alterar) um jogo) e, para esse jogo, editar os valores associados. Quando está a alterar não pode alterar valores de chaves estrangeiras e primárias.

Indicar para cada botão qual o SP que deverá ser executado

Botão: CriarJogo -> criar_jogo()

Botão: Guardar(EditarJogo) -> alterar_jogo(id_jogo, descrição)

Botão: IniciarJogo(Os teus jogos) -> startGame()

Login

Utilizador:

Password:

Login

This is a design stub, functionality not implemented.



Criar Jogo

Descrição:

Criar Jogo

Please fill out this field.

[Cancelar](#)

This is a design stub, functionality not implemented.

Editar o Jogo

Descrição:

Descrição

Jogador:

Player1

Data de início:

2023-10-01 12:00:00

Estado:

Inativo

Guardar

[Cancelar](#)



Os teus jogos

Descrição	Data de início	Estado	Ação
Game 1 Description	2023-10-01 12:00:00	Inativo	Edit
Game 2 Description	2023-10-02 14:30:00	Inativo	Edit
Game 3 Description	2023-10-02 14:30:00	Pronto para começar	Edit
New Game	Start		

This is a design stub, functionality not implemented.



2. Implementação

*Esta secção é para ser preenchida pelo grupo que recebeu o documento. Aqui vão falar da implementação que fizeram. A implementação é o “best of”, ou seja, o que **agora** acham que é a melhor solução, com base em tudo o que aprenderam (com as vossas experiências, com as ideias do outro grupo, discussões com o professor, google, chatgpt, etc), no limite podem não seguir nada do que tinham especificado no documento que entregaram ao outro grupo.*

2.1. Migração Dos Dados

2.1.1. Criação da BD na MongoDB

Versão	Número Réplicas	Número Coleções
Especificação inicial	3	2
Recebida outro grupo	1	4
<u>Implementada</u>	3	4

Table 1 - resumo das réplicas e nº de coleções especificados e implementados

Analisando inicialmente o número de réplicas, decidimos manter com a especificação inicial com 1 cluster com 3 servidores (1 primário e 2 secundários) garantindo assim uma maior tolerância a falhas em caso de falha no servidor primário.

Quanto ao número de coleções, foi recebido pelo outro grupo +2 coleções em relação ao que tínhamos implementado inicialmente sendo estas **“raw_messages”** e **“failed_messages”**, que na qual decidimos descartar as **“raw_messages”** pois seria ocupar memória / espaço para dados que seriam repetidos, mas mantendo as **“failed_messages”** com a alteração em alguns campos.

Depois sobre as coleções que contêm a informação sobre o tópico Move e Sound (**“Move”** e **“Sound”**) decidimos manter as nossas como estavam, pois, com o ID para cada documento no MongoDB permite diferenciar cada mensagem e também evitar duplicações depois ao inserir no MySQL.

Decidimos também retirar o IDGame do MongoDB visto que os executáveis foram mudados e que consequentemente fazia um mau funcionamento ao calcular o IDGame (a partir do Mongo), mas que mantivemos o registo dos últimos IDMove e IDSound numa coleção **“LastIDs”** visto que como é feita 3 réplicas acaba por ser mais seguro do que se tivéssemos simplesmente um ficheiro .txt com os três últimos IDs (maior risco de ser apagado).

Em suma foram implementadas 3 réplicas que vão conter 4 coleções: **“Failed”, “Move”, “Sound” e “LastIDs”**, cujo campos serão os seguintes:



Coleção: **Move**

```
{
  "_id": {
    "$oid": "67d84e150578d7e58112f0c3"
  },
  "IDMove": 1,
  "Player": 15,
  "Marsami": 24,
  "RoomOrigin": 5,
  "RoomDestiny": 7,
  "Status": 1,
  "Hora": "2025-03-17 16:30:13.439964"
}
```

Coleção: **Sound**

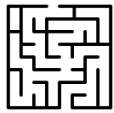
```
{
  "_id": {
    "$oid": "67d85960c54f29acd097e942"
  },
  "IDSound": 1,
  "Hour": "2025-03-17 17:18:24.409075",
  "Player": 15,
  "Sound": "19.04"
}
```

Coleção: **Failed**

```
{
  "_id": {
    "$oid": "67d85960c54f29acd097e942"
  },
  "IDMessage": 1,
  "Timestamp": "2025-03-17 17:18:24.409075",
  "Message": "{ 'Player':17, 'Marsami':21, 'RoomOrigin': 0, 'RoomDestiny': 0, 'Status': 0, 'isMigrated': false, 'Hora': '2025-04-24 12:38:43.612275' }",
  "Report": "1. Colecao Desconhecida"
  "Collection": "Move"
}
```

Coleção: **LastIDs**

```
{
  "_id": {
    "$oid": "67d85960c54f29acd097e942"
  },
  "LastIDMove": 1,
  "LastIDSound": 1
}
```



2.1.2. Descrição Geral do Procedimento de Mongo Para Mysql

Na checkbox da esquerda indicam o que especificaram inicialmente, na do meio a especificação do outro grupo e na da direita a vossa implementação.

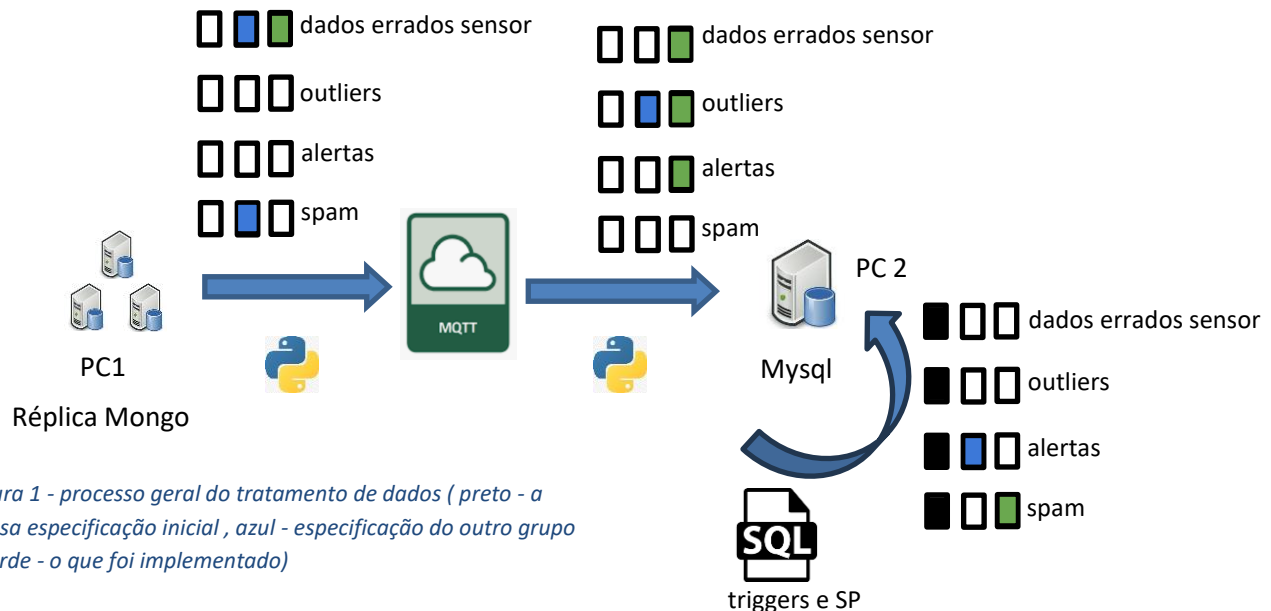


figura 1 - processo geral do tratamento de dados (preto - a nossa especificação inicial , azul - especificação do outro grupo , verde - o que foi implementado)

2.1.2.1. Da Cloud para o Mongo

Da cloud para o Mongo mantivemos as duas threads `mqtt_thread_move` e `mqtt_thread_sound` (explicadas na nossa 1ª parte do relatório) e retirámos a `new_game` thread, visto que já verificamos o término do jogo no código do MQTT para o MySQL (contamos o número de marsamis com status == 2 e status == 0 que têm de ser no total o dobro numbermarsamis [60]). Decidimos também deixar a `check_thread` mas que poderá ser retirada visto que serve somente para monitorizar as mensagens recebidas e inseridas na MongoDB.

De resto não houve alterações durante esta fase.

2.1.2.2. De Mongo para MQTT

Inicialmente não verificávamos os dados errados do Mongo para MQTT pois as mesmas eram feitas por triggers antes de ser inserido no MySQL. Contudo com a ideia de termos adicionado a coleção `"failed_messages"` decidimos que era melhor serem tratados antes de serem subscritos aos tópicos do MQTT.

Também reparámos que o outro grupo decidiu tratar do Spam, mas que nós decidimos só tratar no ponto a seguir (2.1.2.2), visto que os alertas estão mais relacionados com o "Som" e cujo mesmos são tratados a seguir.



Decidimos também verificar de uma maneira inicial alguns dados “Sound” e “Move” (dados que usam os valores do MySQL “Maze”) que vão ser explicados na parte de Tratamento de dados anómalos.

Periodicidade que vai buscar ao Mongo

Decidimos usar uma periodicidade de 0.5 segundos enquanto o outro grupo decidiu utilizar 0.25, decidimos usar 0.5 segundos de maneira a haver tempo na parte da validação antes de ser inserido no tópico MQTT associado ao tipo de dado.

Como é evitado o envio de duas vezes para o MQTT

Decidimos evitar o envio duplicado de mensagens para o MQTT a partir de IDs únicos criados quando estes são inseridos no Mongo, enquanto o outro grupo decidiu fazer recorrendo ao uso do QoS = 2. Previamente já tiramos duplicados que cheguem da Nuvem para o Mongo a partir da Hora que não poderá ser igual em qualquer mensagem em cada coleção.

Número de Threads utilizado

O número de threads utilizado foi de 2 threads em 1 só main. Onde 1 thread foi utilizado para subscrever ao tópico de Move e o outro para o tópico de Sound. O outro grupo decidiu recorrer a 4 threads onde 2 são para receber e publicar mensagens com o tópico “Move” e outras 2 para receber e publicar no tópico de “Sound”, mas mesmo assim decidimos tratar o receber e publicar em 1 só Thread, evitando assim complexidade.

Uso do QoS

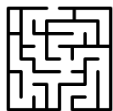
Optou-se por subscrever todos os tópicos com QoS 2, garantindo a entrega exata e ordenada das mensagens. No caso dos tópicos *sound_grupo15* e *move_grupo15*, esta escolha assegura que o *bot* reage de forma adequada aos alertas sonoros e executa corretamente as ações previstas em cada sala.

Relativamente aos tópicos *failed_grupo15* e *ack_grupo15*, apesar de não ser crítica a perda ocasional de mensagens (nomeadamente em *failed_grupo15*, onde a perda representa apenas a omissão de um registo de log), a subscrição com QoS 2 contribui para a fiabilidade global do sistema. No caso específico de *ack_grupo15*, mesmo que a mensagem não seja recebida, o módulo *mqttToMySQL* valida posteriormente a confirmação.

Importa ainda salientar que a receção do tópico *ack_grupo15* permite confirmar que uma mensagem previamente enviada nos tópicos *move_grupo15* ou *sound_grupo15* foi devidamente processada e inserida na base de dados, sendo o respetivo campo *isMigrated* atualizado para *True*.

Nota Técnica: Justificação da utilização de find() em detrimento de watch():

Optou-se por manter a utilização do método *find()* para a consulta de dados, uma vez que este permite a aplicação de validações individualizadas a cada documento recuperado, garantindo posteriormente a sua confirmação através de mensagens de *acknowledgement* (ACK). Este processo não opera em *tempo real*, ao contrário do método *watch()* (baseado em *change streams*), que reage de forma contínua às alterações na base de dados, o que comprometeria a execução das validações e o envio fiável dos ACKs.



Adicionalmente, o uso de `find()` contribui para uma maior estabilidade do sistema ao evitar *bursts* inesperados. O mecanismo de *change streams* envia eventos assim que ocorrem alterações, podendo originar picos de mensagens que sobrecarregam o sistema ou o *broker* MQTT. A utilização de *tokens* no contexto de *streams* também pode introduzir comportamentos indesejados ou inconsistentes.

Por outro lado, a aplicação de um índice sobre o campo `IsMigrated` permite que as consultas `find()` sejam praticamente instantâneas, mesmo em coleções de grande dimensão.

Importa ainda referir que a funcionalidade `watch()` requer, por norma, uma configuração de replicação com pelo menos três *réplicas*, enquanto a abordagem baseada em `find()` pode ser executada numa única instância de *réplica*, tornando-se assim mais adequada a ambientes de desenvolvimento ou de menor escala.

Versão	Periodicidade vai buscar Mongo	Como evitam enviar duas vezes para MQTT	Nº Threads	QOS
Especificação inicial	1	atribuir IDs, usando QoS = 2	2	2
Recebida outro grupo	0.25	Usando QoS = 2 com Streaming	4	2
Implementada	0.5	atribuir IDs, Acks de Confirmação e verificação das datas	2	1 ou 2 (depende do tópico)

tabela 2 - resumo das ações feitas do Mongo para Mqtt

2.1.2.3. De MQTT para MySQL

Optou-se por manter, por conveniência e continuidade documental, a maioria das descrições apresentadas no relatório anterior, sendo novamente incluídas nesta secção as respetivas teorias de base, de forma resumida.

Esta componente é responsável por subscrever os tópicos MQTT previamente definidos e, em seguida, persistir a informação recebida nas respetivas tabelas da base de dados MySQL. Cada tópico é agora tratado através de uma thread pool e de um único processo principal (main thread), garantindo uma escuta contínua e eficiente dos dados provenientes de ambos os sensores em paralelo. Este mecanismo encontra-se descrito de forma mais detalhada nas descrições do código em baixo.

Durante o processamento, são igualmente tratados casos de dados inválidos e também outliers, conforme descrito nas secções 2.2.1 e 2.2.2.

Para prevenir a duplicação de registos na migração de dados do MongoDB para o MySQL, são utilizados os campos `IDSound` e `IDMove` como identificadores únicos. Estes campos, configurados como auto-increment no MongoDB (ver ponto 1.1.2), assumem no MySQL a função de chaves primárias (primary keys),



tendo sido desativado o auto-increment. Esta configuração impede a inserção de registos duplicados, uma vez que a violação da unicidade das chaves primárias gera erro de integridade.

No relatório anterior, os erros de escrita eram registados numa tabela de alertas. No entanto, este procedimento foi posteriormente descontinuado, por se verificar que a tabela não tinha sido concebida para esse fim.

No início de cada jogo, com o arranque do bot, é realizada a sincronização da base de dados local com a instância na cloud, assegurando o carregamento de dados atualizados. Este procedimento encontra-se descrito na secção 1.3.1, referente às alterações ao modelo relacional.

Mantém-se a opção pela utilização do nível de (QoS) 2 na transmissão de mensagens via MQTT, de forma a assegurar a entrega garantida e **ordenada** das mensagens. Esta escolha justifica-se pela necessidade de fiabilidade na receção dos dados por parte do *bot*, uma vez que estes são essenciais para o correto funcionamento e tomada de decisão durante o jogo.

Descrição das principais funcionalidades do código:

Estrutura inicial:

Aquando da execução do código, são estabelecidas as ligações à base de dados local, à base de dados na *cloud* e ao *broker* MQTT. Caso a base de dados local não esteja acessível no momento da inicialização, o processo é interrompido, sendo solicitada a intervenção para restabelecer a ligação. Adicionalmente, é verificada a conectividade com a base de dados na *cloud*; na eventualidade de esta não estar disponível, o sistema prossegue a execução utilizando valores predefinidos.

Mecanismo Keep-alive

O script implementa uma thread independente responsável por verificar, a cada 2 segundos, a disponibilidade da base de dados local. Caso a ligação esteja indisponível, a emissão da mensagem keep-alive é interrompida, iniciando-se automaticamente o processo de reconexão à base de dados local.

Se o script for interrompido ou terminar inesperadamente, a cessação das mensagens keep-alive permite que a instância do MongoDB detete a falha e reaja em conformidade, suspendendo temporariamente o envio de mensagens. Esta abordagem evita a geração excessiva de tráfego no broker e assegura que interrupções momentâneas de serviço não comprometem o funcionamento contínuo do sistema.

Exemplo de mensagem **keep-alive**:

```
{
  "IDMongo": id_sound,
  "collection": "Sound"
}
```



Número de Threads utilizado

As mensagens dos tipos *Sound* e/ou *Move* são tratadas de forma assíncrona, recorrendo a uma *queue* de mensagens que assegura a sua ordenação e integridade antes do processamento. Esta estratégia previne a perda de mensagens em situações de elevada carga.

O sistema recorre a uma *thread pool* (`ThreadPoolExecutor`) com uma capacidade inicial de quatro *threads*, valor que poderá ser ajustado de acordo com as necessidades da aplicação.

Adicionalmente, é utilizada uma *thread* dedicada à receção de mensagens do tipo *keep-alive*, conforme descrito anteriormente, bem como uma *thread* responsável pela inserção de mensagens numa *queue* partilhada.

Esta configuração permite o processamento concorrente das mensagens, promovendo uma maior eficiência, desempenho e escalabilidade do sistema.

Versão	Nº Threads
Especificação inicial	2
Recebida outro grupo	2
<u>Implementado</u>	(Threadpool com 4) + 2

tabela 3 - número de threads utilizadas do Mqtt para MySQL

Escrita na Base de Dados e Mecanismo de Acknowledgement (ACK)

Após o processamento e validação das mensagens, o *script* procede à sua inserção na base de dados. Caso a operação seja concluída com sucesso, é enviada uma mensagem de confirmação (*acknowledgement*, ou *ACK*) para o MongoDB. Em caso de falha na inserção, esta confirmação não é emitida, permitindo ao MongoDB reenviar a mensagem até que a operação seja concluída com êxito.

Situações de tentativa de inserção duplicada podem ocorrer, nomeadamente quando o *ACK* anterior se perde e o MongoDB reenvia a mesma mensagem. Este mecanismo de *acknowledgement* garante, assim, a integridade do processo de comunicação, prevenindo situações de repetição indefinida (*loop*) no envio de mensagens por parte do MongoDB.



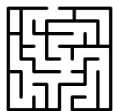
Utilização de um Broker MQTT Privado

Durante o desenvolvimento do nosso projeto, optámos por utilizar um broker MQTT privado, instalado numa máquina virtual (VM) alojada na plataforma Azure, com o serviço Mosquitto. Esta decisão foi motivada por preocupações relacionadas com segurança, fiabilidade e controlo do sistema de comunicação entre os diferentes componentes da aplicação.

Inicialmente, foi utilizado um broker público fornecido pelo professor. No entanto, este tipo de broker apresenta várias limitações, nomeadamente:

- **Acesso aberto a qualquer utilizador**, o que pode resultar em **interferência indesejada** de outros grupos (por acidente ou de forma maliciosa), nomeadamente o controlo de portas ou a receção de mensagens não desejadas;
- **Condições de serviço instáveis**, com possível perda de mensagens devido ser um ambiente de elevada concorrência;

Esta solução permitiu-nos desenvolver e testar o nosso projeto com maior confiança e estabilidade, garantindo que apenas os nossos dispositivos comunicam com o servidor e que as mensagens são entregues de forma fiável e previsível.



2.2. Tratamento dos Dados

2.2.1. Tratamento de dados anómalos (valores de sensores errados)

Descrição

Para tratamento de dados anómalos decidimos separar em **duas fases de tratamento**, **uma do Mongo para MQTT** e **outra de MQTT para MySQL**, ou seja, se a cada valor de Sound ou Move não passar em cada uma destas validações simplesmente é enviada para o tópico “failed_grupo15” e inserida na coleção ‘failed’ se for do Mqtt para MySql, caso contrário é inserida diretamente nessa coleção, posteriormente é apagada na Coleção ‘Move’ ou ‘Sound’ (pois o valor já ficará guardado em outra coleção), as validações foram as seguintes:

1ª fase - Tratamento de dados anómalos [Mongo para MQTT]:

Move	
Validação nº	descrição
1	Verificação se os tipos de variáveis estão corretos
2	Verificação se o número de marsamis são válidos [se não é <0 ou >numbermarsamis]
3	Verificação do formato da Hora
4	Verificação se não há datas duplicadas
5	Verificação se os Rooms são válidos [se não é <0 ou > numberrooms]
6	Verificação se o movimento é válido [baseado na tabela ‘Corridor’ do MySQL da nuvem]

tabela 4 - 1ª fase de tratamento de dados anómalos de Move

Sound	
Validação nº	descrição
1	Verificação do formato da Hora
2	Verificação se o número Sound está no formato correto (float)
3	Verificar se o valor de Sound é um número inteiro (se necessário)
4	Verificação do valor Sound não é negativo
5	Verificação se não há datas duplicadas

tabela 5 - 1ª fase de tratamento de dados anómalos de Sound



2ª fase - Tratamento de dados anómalos [MQTT para MySQL]:

Move	
Validação nº	descrição
1	Sala Origem = 0 e Sala Destino > 0 e status = 1
2	Sala Origem = 0 e Sala Destino = 0 e status = 0 ou status = 2
3	Sala Origem > 0 e Sala Destino > 0 e status = 1
4	Verificação da data do sensor se corresponde à data atual com threshold (5 minutos)

tabela 6 - 2ª fase de tratamento de dados anómalos de Move

Sound	
Validação nº	descrição
1	Verificação de outliers [explicado no tópico a seguir]
2	Verificação da data do sensor se corresponde à data atual com threshold (5 minutos)

tabela 7 - 2ª fase de tratamento de dados anómalos de Sound

Justificações

Resolvemos adicionar mais algumas validações que achamos pertinentes para tornar o código mais “resistente”.

Também foi decidido ao contrário da nossa especificação inicial tratá-los na parte do código do que simplesmente antes de ser inserido de forma a reduzir a complexidade se fosse implementado com Triggers

O motivo de termos separado por fases estas validações provém da maneira como obtemos os valores de modo a fazer as respetivas validações. Ou seja, na 1ª fase de validações recorreremos ao uso da base de dados (BD) ‘Maze’ onde somente fazemos as validações que recorrem ao uso dos valores dessa BD no PC1 (quando for necessário) e que na 2ª fase de validações recorreremos ao uso de uma tabela de constantes chamada ‘Configs’ na nossa BD (“pisid_sql”) que corre no PC2, com isto conseguimos ter uma abordagem que evita o uso de constantes no código em python



2.2.2. Tratamento de outliers de ruído

Descrição

Resolvemos fazer o **tratamento dos outliers de som de Mqtt para Mongo** cujo **abordagem** foi se o **valor médio** [média é feita no máximo com os últimos 4 valores Sound] do som **variar mais do que 3 vezes o desvio padrão** é considerado outlier e posteriormente enviado pelo tópico “failed_grupo15” e **inserido na Coleção ‘Failed’ do Mongo e apagadas da Coleção ‘Sound’**, foi implementada da seguinte forma:

(I) (No início de cada Jogo)

- (1) Ir buscar o valor **noisevartoleration** da tabela “setupmaze” da BD ‘maze’
 - NOISEVARTOL = 2.5
- (2) Ir buscar os valores da tabela “configs” da BD ‘pisid_sql’ estes que vão ser:
 - LIMITE_DESVIO_PADRAO = 3
 - QTD_VALS_SOUND_MAX = 4
 - QTD_VALS_SOUND_MIN = 1

(II) (A cada valor som passado pelo sensor)

- (3) **Se não houver valores de Som** na tabela Sound na BD ‘pisid_sql’ então não é considerado outlier , pois é o primeiro valor a ser inserido ($\text{len}(\text{historico}) < \text{QTD_VALS_SOUND_MIN}$)
- (4) **Se houver mais do que 1 valor de Som e menos do que 4 valores de Som** na tabela Sound na BD ‘pisid_sql’ ($\text{len}(\text{historico}) \geq \text{QTD_VALS_SOUND_MIN}$) então faz a média com esses valores e verifica se é maior que o limite ($\text{NOISEVARTOL} * \text{LIMITE_DESVIO_PADRAO}$) ignorando os valores que poderiam ser outliers visto que se trata ainda do início do jogo evitando assim falsos positivos
- (5) **Se houver mais do que 4 valores de Som**, então faz sempre a média a cada valor novo de som com os últimos 4 valores inseridos na tabela Sound na BD ‘pisid_sql’ e vai verificando se a diferença do valor atual do som - média não ultrapassa o limite

Justificação

Também foi decidido ao contrário da nossa especificação inicial tratá-los na parte do código do que simplesmente antes de ser inserido de forma a reduzir a complexidade se fosse implementado com Triggers e também por ter sido uma ideia que tirámos do outro grupo de tratá-los de Mqtt para MySQL.

O motivo de ter sido utilizado 3 vezes o desvio padrão foi porque este realmente é um método de verificação de outliers conhecido como *Z-Score* ao contrário da implementação do outro grupo que acabava



por ter grande sensibilidade ao detetar outliers visto que a variação (1 vez) é menor e que simplesmente usando o valor atual e não o “histórico”

2.2.3. Tratamento de Alertas de Som

Alertas de Sons

Descrição

A lógica consiste em verificar se o valor atual de som está a 80% do valor limite que consideramos o valor de $\text{normalnoise} + \text{noisevartoleration} = 21.5$. Se tiver é mandado uma mensagem para a tabela ‘mensagens’, posteriormente este se atingir 90% do valor anteriormente referido então é enviado uma mensagem para a tabela ‘mensagens’ e depois é acionado o atuador de fechar todas as portas.

Justificação

No tratamento dos Alertas de Som, decidimos adotar os mesmos valores percentuais para verificar um valor de som bastante elevado, mas dando mais ênfase ao valor limite do maze em si (valor limite que depois pode ser alterado). Assim, implementámos a lógica diretamente no código que comunica via MQTT para MySQL, permitindo o acesso direto ao limite de tolerância do som (noisevartoleration).

SPAM de Mensagens (de Som)

Descrição

Para o tratamento da inserção demasiado frequente de alertas de som na tabela ‘mensagens’ recorreu-se ao uso de um valor (que é preenchido ao criar cada jogo) chamado por spamVal que é o valor que representa o tempo até outra mensagem poder ser inserida na tabela ‘mensagens’, caso contrário é rejeitada e não é inserida.

Esta verificação é feita com o uso de um trigger before insert na tabela mensagens que vai verificando a diferença entre o último campo ‘HoraEscrita’ e o campo ‘HoraEscrita’ a ser inserido que deverá ser maior que o spamVal.

Justificação

O outro grupo no relatório não acabou por abordar a maneira como seriam tratados o SPAM, mas nós decidimos usar em trigger pois não acaba por haver sempre um uso recorrente desta ação e onde também não precisamos de dar SELECT no último valor, fazer a verificação em código e depois sim dar INSERT.



2.2.4. Tratamento de número de marsamis numa sala (obter pontuação)

Descrição

Foram desenvolvidos dois tipos de *scripts* (bots), ambos baseados em dados tratados previamente na base de dados SQL. Estes *scripts* realizam consultas sobre a tabela 'ocupacaolabirinto', garantindo que operam com informação atualizada e consistente, prevenindo, assim, a utilização de dados inválidos. Segue-se a descrição detalhada do funcionamento de cada um:

Solução 1:

A presente solução é exposta de forma concisa, dado que a descrição detalhada se mantém essencialmente idêntica à do relatório anterior.

Este *script* verifica, para cada sala, se o número de entidades do tipo *Marsami* com paridade Odd e Even é igual, consultando diretamente a tabela ocupacaolabirinto. Quando esta condição é satisfeita, o *script* executa uma tentativa de pontuação na sala e aguarda um segundo antes de prosseguir, de forma a evitar desperdício de tentativas consecutivas.

Adicionalmente, o *script* monitora periodicamente a tabela mensagens para detetar alertas relacionados com níveis de ruído excessivos no labirinto. Caso um aviso seja identificado, o *script* aciona o fecho de todas as portas durante aproximadamente cinco segundos, com o objetivo de reduzir o ruído ambiente, reabrindo-as após esse intervalo.

Solução 2

Esta abordagem utiliza um dicionário para armazenar os movimentos possíveis entre salas, com base na informação contida na tabela corredor. Inicialmente, todas as "portas" são consideradas abertas.

O *script* executa três *threads* paralelas, permitindo uma reação rápida a alterações no estado do labirinto:

- **control_individual_door()**

A função tem como objetivo analisar e regular o desequilíbrio entre entidades *Marsami Odd* e *Marsami Even* em cada sala do labirinto.

Em intervalos regulares (definidos por `CHECK_INTERVAL`), a função consulta a tabela ocupacaolabirinto, que regista a quantidade de *Marsamis* de cada paridade presentes em cada sala.

Com base nesta informação, são identificadas todas as portas cuja origem corresponde à sala em análise, sendo avaliado o seu estado atual (aberta ou fechada).

Se for detetado um desequilíbrio relevante, a função verifica se alguma sala adjacente possui uma maior concentração de *Marsamis* do tipo necessário para restabelecer o equilíbrio. Nesses casos,



a porta respetiva é aberta, permitindo a movimentação. Caso contrário, a porta é fechada para evitar um agravamento do desequilíbrio.

- **check_score_trigger()**

Executada numa *thread* separada, esta função verifica a existência de salas com igual número de *Marsamis Odd* e *Even*. Se a condição for satisfeita, são enviadas até três mensagens de pontuação por sala. Foi implementado um sistema de contagem para evitar disparos infinitos.

- **check_sound_limit()**

Também executada numa *thread* dedicada, esta função consulta a tabela mensagens para identificar alertas sonoros. Caso sejam detectados, todas as portas são temporariamente fechadas, conforme descrito anteriormente.

O uso de *threads* permite que o bot monitorize e reaja em tempo real ao estado do jogo, sem comprometer outras funcionalidades.

Adicionalmente, em ambos os scripts, existe uma *thread* suplementar que, a cada segundo, verifica se existe um jogo em execução. Caso não exista, ambos os *scripts* são terminados de forma controlada.

Esta lógica justifica-se pelo facto de a interface gráfica (GUI), através da qual o jogador interage, solicitar explicitamente a execução destes *scripts*. Desta forma, assegura-se que o *bot* apenas opera quando um jogo se encontra no estado *running*, evitando a sua execução contínua ou indevida fora de contexto.

Justificação das Abordagens Implementadas

Foram exploradas duas abordagens distintas para alcançar a funcionalidade de pontuação. Embora uma estrutura em grafo tenha sido inicialmente considerada, optou-se pela utilização de um dicionário, considerado uma estrutura funcionalmente equivalente neste contexto. Esta escolha permitiu uma implementação mais simples e eficiente, sem comprometer a lógica de navegação entre salas.



2.2.5. Implementação de Stored Procedures SQL de apoio à migração e tratamento de dados

É nesta tabela que deverão ser listados os SP que implementam mecanismos anteriores. Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) – Alterado com base em ideia de outro grupo

A(ni) – Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) – Novo com base em ideia novas

A – Alterado com base em novas ideias

Nome SP	Argumentos	Descrição	

Justificação

No relatório anterior, foi incluído um procedimento armazenado (*stored procedure*) designado `getHighScore`, o qual foi posteriormente removido. Esta decisão deveu-se à existência de uma funcionalidade de pesquisa integrada no *website*, tornando a referida *stored procedure* redundante. Adicionalmente, esta funcionalidade não se enquadra nos critérios estabelecidos para mecanismos de apoio ao processo de migração de dados.

Importa referir que a mesma confusão foi aparentemente partilhada por outro grupo, pelo que não foram extraídas contribuições relevantes.

No contexto do nosso projeto, conforme detalhado ao longo do presente relatório, as operações em questão são executadas através de *triggers* ou implementadas diretamente em código Python.



2.2.6. Implementação de Triggers

É nesta tabela que deverão ser listados os triggers Na sexta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) – Alterado com base em ideia de outro grupo

A(ni) – Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) – Novo com base em ideia novas

A – Alterado com base em novas ideias

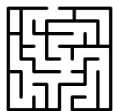
Nome Trigger	Tabela	Tipo de Operação (I,U,D)	Evento (After, Before)	Notas	
after_setupmaze_insert	SetupMaze	Insert	After	A sua principal função é inicializar as salas na tabela labirinto para o jogo que se encontra em estado <i>running</i> . Garantido assim que a tabela <i>ocupacaolabirinto</i> exibe todas as salas que existem no labirinto.	N(ni)
after_setupmaze_update	SetupMaze	Insert	After	A sua principal função é atualizar as salas na tabela <i>ocupacaolabirinto</i> para o jogo que se encontra no estado <i>running</i> . Remove as salas antigas e insere novamente todas as salas com base no número atualizado de salas, garantindo que a tabela reflita sempre as configurações atuais do labirinto.	A(og)
after_medicoesp_assagens_insert	medicoesp assagem	Insert	After	O objetivo deste trigger é atualizar corretamente o número de marsamis Odd e Even na tabela <i>ocupacaolabirinto</i> . Refletido assim os movimentos no labirinto	N(ni)
before_insert_mensagens_spam_check	mensagens	Insert	Before	A sua principal função é evitar spam na inserção de novas mensagens. Garante que não sejam registadas mensagens duplicadas num curto espaço de tempo.	A

tabela 8 – Triggers implementados

Justificação

O número de *triggers* foi reduzido, uma vez que determinadas operações passaram a ser executadas em código, por se considerar mais adequada a sua análise e manutenção.

Adicionalmente, considerou-se pertinente a definição de um *trigger* associado às operações de *INSERT* ou *UPDATE* na tabela SetupMaze, com o objetivo de inicializar todas as salas com o valor zero. Esta abordagem



visa garantir que, caso os *Marsamis* não se desloquem a todas as salas, estas não fiquem omissas na base de dados nem na aplicação Android.

Considerou-se ainda mais eficiente atualizar automaticamente a posição dos *Marsamis* na tabela *ocupacaolabirinto* através de um *trigger*, acionado sempre que é inserida uma nova mensagem na tabela *medicoespasagem*.

Por fim, foi implementado um *trigger* adicional para controlar o número de mensagens recebidas na tabela *Mensagens*, permitindo ao jogador configurar o limite de alertas que pretende receber, conforme definido no campo *SpamTol*.

2.3. Base de Dados

2.3.1. Modelo Relacional

Diagrama relacional completo implementado. Assinalar a azul alterações derivadas de outro grupo e a verde alterações novas.

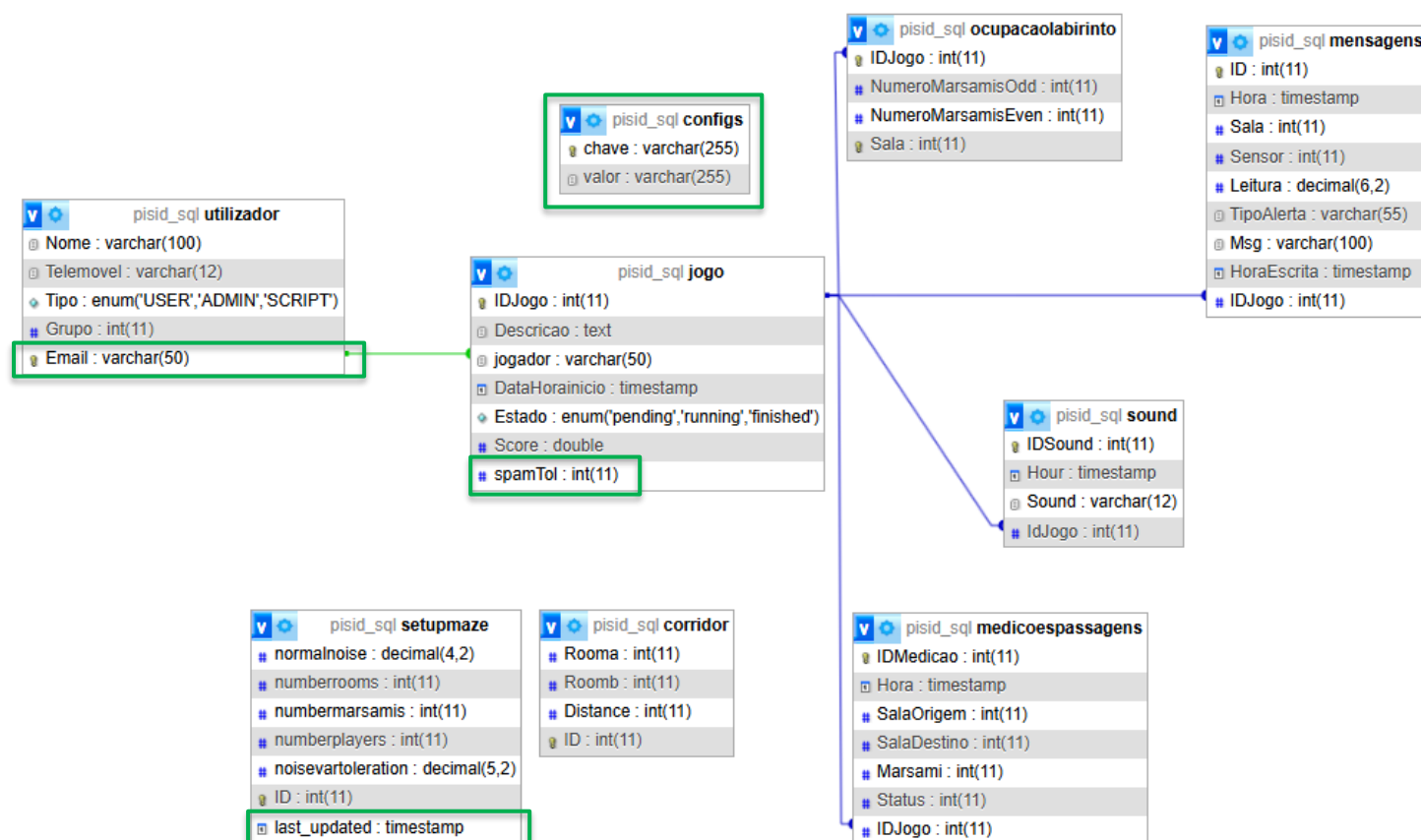


Figura 2 – base de dados MySQL implementada (verde – o que foi adicionado na BD)

Para garantir a continuidade documental e por questões de conveniência, optou-se por reutilizar, de forma sintetizada, as principais descrições e teorias apresentadas no relatório anterior.



Tabela MedicoesPassagens e Sound

Optámos por não definir os campos IDSound e IDMedicoes como auto_increment. Tal decisão foi tomada para evitar a inserção de dados duplicados.

Adicionalmente, incluímos o atributo IDJogo nas tabelas MedicoesPassagens, Mensagens e Sound. Com esta abordagem, é possível verificar, a cada inserção, se existe um jogo em curso, uma vez que não faz sentido registar dados nestas tabelas se não houver um jogo associado.

As relações entre estas tabelas foram definidas da seguinte forma:

ON UPDATE RESTRICT: Esta configuração evita problemas quando o jogo ainda não existe, garantindo que o incremento dos IDs não resulte em conflitos de chaves primárias duplicadas.

ON DELETE CASCADE: Esta opção elimina automaticamente os dados relacionados ao registo principal quando ele é apagado, uma vez que essas informações não fazem sentido sem o jogo. Utilizar o SET NULL para preservar os dados para fins de debug dificultaria a análise, pois as relações entre os registos seriam perdidas.

Tabela Jogo

Foram ainda adicionados dois novos atributos à tabela Jogo: Estado e Score. O campo Estado permite identificar a fase em que o jogo se encontra, podendo assumir **três valores**:

- **Pending** – O jogo foi criado, mas à espera de correr.
- **Running** – O jogo está a decorrer.
- **Finished** – O jogo terminou com sucesso.

O campo Score serve para quando um jogo termina o Score total desse mesmo jogo é atualizado na tabela 'jogo'.

Tabela configs

Tabela que acaba por representar as constantes que usámos no código, com o objetivo de evitar constantes pelo código

Justificação

Tabela Jogo

Em comparação com o grupo 33 e a nossa Parte 1 foi feita simplesmente 2 alterações nesta tabela: **(1)** as mudanças descritas na descrição, **(2)** adicionou-se o campo spamTol, que representa o tempo entre mensagens (é definido pelo user) a serem inseridas na tabela 'mensagens', **(3)** mudou-se o campo do Estado de varchar para Enum, visto que para o âmbito deste projeto o jogo simplesmente tem 3 estados possíveis.



Tabela Utilizador

Só mudámos a Primary key para o email que é como o grupo 33 implementou e também porque era o que estava no enunciado (nós inicialmente tínhamos o Grupo como Pk) e também alterámos o tipo de utilizar para um enum com os valores USER , ADMIN e SCRIPT

Tabela setupmaze e corredor

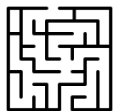
Tabelas cujo o outro grupo não implementou na sua base de dados , mas que nós decidimos manter visto que são usadas pelo Android.

Tabela ocupacaoLabirinto

Manteve-se igual à nossa Parte 1 e da Parte 1 do grupo 33

Extra

Foi retirada a nossa tabela “TipoAlerta”, pois trazia mais complexidade para os tipos de alerta que temos atualmente que são quando o valor de som é um outlier, representa 80% do valor limite e representa 90% do valor limite



2.3.2. Utilizadores Base de Dados MySQL

Nesta secção deverão ser indicados os utilizadores e perfis implementados (têm de consta todos os SP usados). Assinalar a azul alterações derivadas de outro grupo e a verde alterações novas.

Tabela	Tipo de Utilizador			
	Admin BD (root)	Admin App	Jogador	Script
Utilizador	U/I/D/L	U/I/D/L	-	-
Jogo	U/I/D/L	U/I/D/L	L	U/L
Mensagens	U/I/D/L	U/I/D/L	L	I
Sound	U/I/D/L	U/I/D/L	L	I
MedicoesPassagens	U/I/D/L	U/I/D/L	L	I
OcupacaoLabirinto	U/I/D/L	U/I/D/L	L	-
Config	U/I/D/L	U/I/D/L	-	-
Corridor	U/I/D/L	U/I/D/L	-	U/I/L
SetupMaze	U/I/D/L	U/I/D/L	-	U/I/L
Stored Proc.				
Criar_utilizador	X	X	-	-
Remover_utilizador	X	X	-	-
Criar_jogo	X	X	X	-
Eliminar_Jogo	X	X	X	-
Alterar_utilizador	X	X	-	-
Alterar_jogo	X	X	X	-
CreateDbUser	X	X	-	-
ExistsGame	X	X	-	-
ExistsUtilizador	X	X	-	-
GetGames	X	X	-	-
Correr_Jogo	X	X	X	-
ValidEmail	X	X	-	-

Em que U = Update, I = Insert, D = Delete, L = Leitura, X = Executar SP e - = sem permissões.

tabela 9 – Utilizadores e permissões diante as tabelas e SPs

Descrição

A gestão de utilizadores da base de dados MySQL do projeto é feita através de roles específicos, garantindo que cada utilizador tem apenas as permissões necessárias para desempenhar as suas funções, melhorando a organização do sistema.

No nosso projeto, foram definidas **4 roles**:



- **Administrador da base de dados:** Tem acesso total a toda a base de dados (tabelas e stored procedures). Pode criar e remover utilizadores, definir permissões e modificar qualquer dado. Para facilitar a implementação, este será representado pelo utilizador root.
- **Administrador da aplicação:** Tem permissões para criar e gerir os jogadores e os seus jogos, bem como a aplicação em si. Nomeadamente, permissões para executar os stored procedures Criar_utilizador, Remover_utilizador, Alterar_utilizador, Criar_jogo, Alterar_jogo e Eliminar_jogo. Como pedido no enunciado, este terá uma forma de reinicializar o processo de migração, mas como esse processo será feito em python, não é necessário tratar dos seus privilégios relacionados com o MySQL.
- **Jogador:** Tem permissões para criar jogos, alterar os seus próprios dados e modificar algumas informações dentro dos seus jogos. Nomeadamente tem permissões de leitura das tabelas da base de dados e de execução dos stored procedures relacionados com os seus dados e o seu jogo, como o Alterar_password, Criar_jogo, Alterar_jogo e Eliminar_jogo.
- **Script:** Tem permissões para inserir novos dados nas tabelas medicoespassagens, sound e mensagens e para ler da tabela Jogo para obter o IDJogo. Adicionalmente também tem permissões para ler, adicionar e alterar dados nas tabelas locais, que foram copiadas da nuvem, Corridor e SetupMaze. O utilizador que vai ter esta role é o *mqtt2mysql*. Decidimos ter Script como role de forma a permitir uma possível expansão futura, por exemplo, caso seja necessário adicionar outro utilizador que represente um script que envie dados ao MySQL através do MQTT.

Além disso, decidimos deixar a gestão das passwords para o próprio MySQL, de forma a garantir mais segurança e evitar a necessidade de armazenar ou manipular estes dados manualmente. O Administrador da aplicação realiza essa tarefa diretamente no MySQL, na tabela *mysql.user*, e posteriormente fornece esses dados ao utilizador.

Justificação

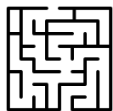
De salientar que a role de Administrador da Base de Dados não foi implementada porque o seu papel era mais de nível figurativo, daí a ter ficado com o root.

Após o feedback recebido na oral, fizemos as seguintes alterações à nossa especificação inicial:

- A role **Admin App** passou a ter os mesmos privilégios do **Admin BD**, facilitando a administração da aplicação sem necessidade de recorrer ao utilizador root.
- Na role **Jogador** foi retirada a permissão de leitura sobre a tabela Utilizador, pois considerámos que um jogador não deve ter acesso a informações de outros utilizadores no sistema.

Estas alterações mantêm a gestão do sistema simples e vão em conta ao que foi sugerido pelo professor na oral intercalar.

Em relação à solução do grupo 33, decidimos não a seguir pelas seguintes razões:



- O utilizador/role **Administrador** tinha apenas permissões para gestão de utilizadores, o que consideramos insuficiente visto que este deve ter acesso total ao sistema.
- O utilizador/role **Jogador** tinha permissões para inserir e editar dados em tabelas críticas, o que de certa forma vai contra a lógica do jogo em si.
- Optámos por não incluir o utilizador/role **Tester** porque não considerámos que tivesse uma função relevante no projeto, visto que teria as mesmas permissões que um jogador normal.

2.4. Aplicação

2.4.1. Procedimentos de Manutenção da Aplicação

É nesta tabela que deverão ser listados os SP que implementam mecanismos anteriores. Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) - Alterado com base em ideia de outro grupo

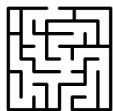
A(ni) - Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) - Novo com base em ideias novas

A - Alterado com base em novas ideias

Nome SP	Argumentos	Descrição	
Criar_utilizador	Nome, Telemovel, Tipo, Email, Password	Cria um utilizador novo na tabela mysql.user do MySQL, onde a password é gerada pelo administrador e é registado na tabela utilizador	=
Remover_utilizador	Email	Remove o utilizador da tabela MySQL, preservando os dados da tabela utilizador com a exceção do Email	=
Criar_jogo	Descrição, spam Tol	É criado um novo jogo que é registado na tabela jogo com uma descrição, tolerância de spam entre mensagens e o respectivo utilizador associado	A
Eliminar_Jogo	IDJogo	É ocultado o registo do jogo para utilizador	=
Alterar_utilizador	Nome, Telemovel, Tipo, Email, Grupo	Altera os campos da tabela utilizador, impossibilitando apenas a alteração dos campos Email por ser chave primária e o campo Grupo	=
Alterar_jogo	IDJogo, Descrição, spam Tol	Alteração da descrição e/ou do score do jogo, só pode ser alterado após o jogo estar finalizado.	A



CreateDbUser	Email, Password	Cria o utilizador novo na tabela mysql.user do MySQL, com os respectivos atributos e permissões de utilizador	N
ExistsGame	IDJogo	Retorna um Boolean que informa se o jogo que se está a tentar alterar de alguma maneira existe ou não	N
ExistsUtilizador	Email	Retorna um Boolean que informa se o utilizador que se está a tentar alterar ou aceder de alguma maneira à base de dados existe ou não	N
GetGames	Grupo	Retorna todos os jogos de um determinado grupo	N
Correr_Jogo	IDJogo	Passa o estado de um dado jogo para 'running'	N
ValidEmail	Email	Retorna um boolean que informa se o email tem um formato válido ou não	N

tabela 10 – SPs implementados

Justificação

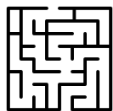
Foram feitas alterações relativamente ao que se tinha pensado implementar no início e não foram adotadas soluções do outro grupo.

Estas alterações tiveram como intuito simplificar o funcionamento das Stored Procedures e também torná-las mais seguras. No caso da Stored Procedure **Criar_jogo** foram adicionados os campos descrição e spamTol, foi ainda utilizada uma abordagem diferente na parte de input, invés de ser utilizado um número fixo de inputs, o tipo de input foi alterado para formato JSON e foram atribuídos valores default tanto ao campo Descricao, como ao campo spamTol, a descrição serve apenas para o utilizador dar uma descrição inicial ao jogo se assim o entender e o spamTol define o intervalo de tempo mínimo entre mensagens para que não sejam consideradas spam.

A Stored Procedure **Alterar_jogo** foi igualmente alterada, visto que esta antes recebia o IDJogo, Descrição, DataInicio e o Score e, no entanto, não é necessário passar estes campos todos e torna-se redundante, uma vez que estes não podem ser alterados pelo jogador, a alteração feita foi a remoção dos campos DataInicio e o Score, e foi adicionado o campo spamTol, para se poder alterar a tolerância de spam caso o jogo ainda não tenha sido corrido. Nesta Stored Procedure foi também alterada a maneira como damos input, para ser possível usar o formato JSON para alterar as informações, sendo assim possível alterar apenas um dos campos sem ter de passar o outro.

A Stored Procedure **Alterar_utilizador** foi também alterada, tendo sido removido o campo Password e adicionado o campo Grupo. Nesta Stored Procedure, também foi utilizado o formato de input JSON, visto que se quisermos apenas alterar o Nome do utilizador, não é necessário estar a passar todos os outros campos, ou, se quisermos atualizar dois ou mais campos também o é possível.

Foram também criados stored procedures com base em ideias que surgiram durante a implementação. Os SPs **ExistsGame**, **ExistsUtilizador**, **ValidEmail** e **GetGames** foram implementados como funções auxiliares



para validar, respetivamente, a existência de jogos, a existência de utilizadores, o formato de email e para obter todos os jogos associados a um grupo específico. Estes SPs são utilizados por outros para evitar repetição de lógica e garantir maior consistência nos dados.

O **CreateDbUser** foi criado com o intuito de termos uma forma automática para criar utilizadores diretamente na base de dados MySQL, garantindo que cada utilizador da aplicação tem o respetivo acesso configurado.

Por fim, o **Correr_Jogo** foi desenvolvido para permitir a alteração controlada do estado de um jogo para 'running', com as devidas validações de permissões e estado atual do jogo, para efetivamente termos uma representação lógica na base de dados de que o jogo está em progresso.

2.4.2. Eventos de suporte à aplicação (caso existam)

É nesta tabela que eles deverão ser listados todos os eventos relevantes para o processo de migração Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) – Alterado com base em ideia de outro grupo

A(ni) – Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) – Novo com base em ideias novas

A – Alterado com base em novas ideias

Nome Evento	Local Execução (Mysql/Windows)	Muito breve descrição	

Justificação

Contrariamente ao inicialmente previsto, não foi implementado qualquer *evento* no sistema. Esta decisão deveu-se, fundamentalmente, à complexidade associada à sua execução. Um dos requisitos identificados envolvia a colocação de um *script* na pasta de arranque do sistema, com uma função dedicada ao registo da data atual. Adicionalmente, seria necessário configurar permissões específicas para que esse *script* pudesse aceder à base de dados SQL, o que implicaria alterações adicionais.

Paralelamente, considerou-se que a demonstração desta funcionalidade durante a apresentação seria tecnicamente desafiante e de utilidade limitada no contexto do projeto.



2.5. Anexos de PrintScreen

[illegible]

Trabalho de Projeto de Integração de Sistemas de Informação Distribuídos



Utilizador: Diogo

Procurar jogo

Detalhe do Jogo #3
Visualize e edite as informações do jogo

Informações

Jogador
player1@example.com

Data de Início
2025-05-08 01:00:24

Descrição
WAWKOROKOKING

Voltar Guardar Alterações

Figura 5 – Página de editar os dados de um jogo

Utilizador: Rafael

Procurar jogo

Criar um jogo novo:

Descrição:

Delay Time entre mensagens: 10 segundos

Utilizador: Rafael

Criar jogo

Figura 6 – Página de criação de um novo jogo



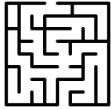
2.5.2. PrintScreen dos gráficos Android com dados



Figura 7 – Gráfico da distribuição de marsamis por sala



Figura 8 – Gráfico do som ao longo do jogo



21:38 segunda, 5/05

61%

Mensagens

Msg	Leitura	Sala	Sensor	TipoMensagem	Hora	HoraEscrita
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:38:39	2025-05-05 21:38:39
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:38:33	2025-05-05 21:38:33
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:38:29	2025-05-05 21:38:29
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:38:16	2025-05-05 21:38:16
Som a 80% do limite	21.19		2	Limite 80%	2025-05-05 21:38:05	2025-05-05 21:38:05
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:38:02	2025-05-05 21:38:02
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:37:50	2025-05-05 21:37:50
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:37:36	2025-05-05 21:37:37
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:37:34	2025-05-05 21:37:34
Som a 80% do limite	21.00		2	Limite 80%	2025-05-05 21:37:24	2025-05-05 21:37:24

Sensor Sound

Procedures

Marsami/Room

Figura 8 – Mensagens de alerta recebidas do Android



2.6. Anexos Códigos MySQL

2.6.1. Código Exportado do phpMyAdmin

```
SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
START TRANSACTION;
SET time_zone = "+00:00";

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Banco de dados: `pisid_sql`
--

--
-- Estrutura da tabela `configs`
--

CREATE TABLE `configs` (
  `chave` varchar(255) NOT NULL,
  `valor` varchar(255) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Extraindo dados da tabela `configs`
--

INSERT INTO `configs` (`chave`, `valor`) VALUES
('datetime_threshold', '5'),
('limite_80', '0.8'),
('limite_90', '0.9'),
('limite_desvio_padrao', '3'),
('num_sala_max', '10'),
('num_sala_min', '0'),
('qtd_valores_sound_max', '4'),
('qtd_valores_sound_min', '2'),
('status_cansado', '2'),
('status_nenhuma_porta', '0'),
('status_tudoOK', '1');

--
-- Estrutura da tabela `corridor`
--
```



```
CREATE TABLE `corridor` (  
  `Rooma` int(11) NOT NULL,  
  `Roomb` int(11) NOT NULL,  
  `Distance` int(11) NOT NULL,  
  `ID` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=latin1 COLLATE=latin1_swedish_ci;  
  
--  
-- Extraindo dados da tabela `corridor`  
--  
  
INSERT INTO `corridor` (`Rooma`, `Roomb`, `Distance`, `ID`) VALUES  
(1, 2, 16, 1),  
(1, 3, 20, 2),  
(2, 4, 20, 3),  
(2, 5, 20, 4),  
(3, 2, 20, 5),  
(4, 5, 20, 6),  
(5, 3, 20, 7),  
(5, 6, 20, 8),  
(5, 7, 20, 9),  
(6, 8, 20, 10),  
(7, 5, 20, 11),  
(8, 9, 20, 12),  
(8, 10, 20, 13),  
(9, 7, 20, 14),  
(10, 1, 20, 15);  
  
-----  
  
--  
-- Estrutura da tabela `jogo`  
--  
  
CREATE TABLE `jogo` (  
  `IDJogo` int(11) NOT NULL,  
  `Descricao` text NOT NULL,  
  `jogador` varchar(50) NOT NULL,  
  `DataHorainicio` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE  
current_timestamp(),  
  `Estado` enum('pending','running','finished') NOT NULL DEFAULT 'pending',  
  `Score` double NOT NULL DEFAULT -1,  
  `spamTol` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
--  
-- Extraindo dados da tabela `jogo`  
--  
  
INSERT INTO `jogo` (`IDJogo`, `Descricao`, `jogador`, `DataHorainicio`, `Estado`,  
`Score`, `spamTol`) VALUES  
(1, '"Jogo fixe 1"', 'joao@gmail.com', '2025-05-10 17:38:13', 'pending', -1, 3),  
(3, '"Jogo fixe 2"', 'joao@gmail.com', '2025-05-10 17:38:43', 'pending', -1, 3),  
(6, '"Jogo fixe 3"', 'joao@gmail.com', '2025-05-10 17:39:03', 'pending', -1, 3),
```



```
(9, '\"Jogo fixe 5\"', 'joao@gmail.com', '2025-05-10 17:39:18', 'pending', -1, 3);

-- -----

--
-- Estrutura da tabela `medicoespassagens`
--

CREATE TABLE `medicoespassagens` (
  `IDMedicao` int(11) NOT NULL,
  `Hora` timestamp NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
  `SalaOrigem` int(11) NOT NULL,
  `SalaDestino` int(11) NOT NULL,
  `Marsami` int(11) NOT NULL,
  `Status` int(11) NOT NULL,
  `IDJogo` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- -----

--
-- Estrutura da tabela `mensagens`
--

CREATE TABLE `mensagens` (
  `ID` int(11) NOT NULL,
  `Hora` timestamp NULL DEFAULT NULL ON UPDATE current_timestamp(),
  `Sala` int(11) DEFAULT NULL,
  `Sensor` int(11) NOT NULL,
  `Leitura` decimal(6,2) NOT NULL,
  `TipoAlerta` varchar(55) NOT NULL,
  `Msg` varchar(100) NOT NULL,
  `HoraEscrita` timestamp NULL DEFAULT current_timestamp(),
  `IDJogo` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Estrutura da tabela `ocupacaolabirinto`
--

CREATE TABLE `ocupacaolabirinto` (
  `IDJogo` int(11) NOT NULL,
  `NumeroMarsamisOdd` int(11) NOT NULL,
  `NumeroMarsamisEven` int(11) NOT NULL,
  `Sala` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- -----

--
-- Estrutura da tabela `setupmaze`
--
```



```
CREATE TABLE `setupmaze` (
  `normalnoise` decimal(4,2) NOT NULL,
  `numberrooms` int(11) NOT NULL,
  `numbermarsamis` int(11) NOT NULL,
  `numberplayers` int(11) NOT NULL,
  `noisevartoleration` decimal(5,2) NOT NULL,
  `ID` int(11) NOT NULL,
  `last_updated` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=latin1 COLLATE=latin1_swedish_ci;

--
-- Extraindo dados da tabela `setupmaze`
--

INSERT INTO `setupmaze` (`normalnoise`, `numberrooms`, `numbermarsamis`,
`numberplayers`, `noisevartoleration`, `ID`, `last_updated`) VALUES
(19.00, 10, 30, 40, 2.50, 0, '0000-00-00 00:00:00'),
(19.20, 10, 30, 40, 2.00, 1, '2025-05-10 15:25:03');

--
-----

--
-- Estrutura da tabela `sound`
--

CREATE TABLE `sound` (
  `IDSound` int(11) NOT NULL,
  `Hour` timestamp NULL DEFAULT NULL ON UPDATE current_timestamp(),
  `Sound` varchar(12) NOT NULL,
  `IdJogo` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-----

--
-- Estrutura da tabela `utilizador`
--

CREATE TABLE `utilizador` (
  `Nome` varchar(100) NOT NULL,
  `Telemovel` varchar(12) NOT NULL,
  `Tipo` enum('USER', 'ADMIN', 'SCRIPT') NOT NULL,
  `Grupo` int(11) NOT NULL,
  `Email` varchar(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Extraindo dados da tabela `utilizador`
--

INSERT INTO `utilizador` (`Nome`, `Telemovel`, `Tipo`, `Grupo`, `Email`) VALUES
('Joao', '910110110', 'USER', 15, 'joao@gmail.com');
```



```
--
-- Índices para tabelas despejadas
--

--
-- Índices para tabela `configs`
--
ALTER TABLE `configs`
  ADD PRIMARY KEY (`chave`);

--
-- Índices para tabela `corridor`
--
ALTER TABLE `corridor`
  ADD PRIMARY KEY (`ID`);

--
-- Índices para tabela `jogo`
--
ALTER TABLE `jogo`
  ADD PRIMARY KEY (`IDJogo`),
  ADD KEY `jogo_ibfk_1` (`jogador`);

--
-- Índices para tabela `medicoespassagens`
--
ALTER TABLE `medicoespassagens`
  ADD PRIMARY KEY (`IDMedicao`),
  ADD KEY `jo` (`IDJogo`);

--
-- Índices para tabela `mensagens`
--
ALTER TABLE `mensagens`
  ADD PRIMARY KEY (`ID`),
  ADD KEY `chaveestrangeira3` (`IDJogo`);

--
-- Índices para tabela `ocupacaolabirinto`
--
ALTER TABLE `ocupacaolabirinto`
  ADD PRIMARY KEY (`IDJogo`, `Sala`),
  ADD KEY `IDJogo` (`IDJogo`);

--
-- Índices para tabela `setupmaze`
--
ALTER TABLE `setupmaze`
  ADD PRIMARY KEY (`ID`);

--
-- Índices para tabela `sound`
--
ALTER TABLE `sound`
  ADD PRIMARY KEY (`IDSound`),
```



```
ADD KEY `chaveestrangeira2` (`IDJogo`);

--
-- Índices para tabela `utilizador`
--
ALTER TABLE `utilizador`
  ADD PRIMARY KEY (`Email`);

--
-- AUTO_INCREMENT de tabelas despejadas
--

--
-- AUTO_INCREMENT de tabela `corridor`
--
ALTER TABLE `corridor`
  MODIFY `ID` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=16;

--
-- AUTO_INCREMENT de tabela `jogo`
--
ALTER TABLE `jogo`
  MODIFY `IDJogo` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=10;

--
-- AUTO_INCREMENT de tabela `mensagens`
--
ALTER TABLE `mensagens`
  MODIFY `ID` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=85;

--
-- Restrições para despejos de tabelas
--

--
-- Limitadores para a tabela `jogo`
--
ALTER TABLE `jogo`
  ADD CONSTRAINT `jogo_ibfk_1` FOREIGN KEY (`jogador`) REFERENCES `utilizador`
  (`Email`) ON DELETE CASCADE ON UPDATE CASCADE;

--
-- Limitadores para a tabela `medicoespassagens`
--
ALTER TABLE `medicoespassagens`
  ADD CONSTRAINT `jo` FOREIGN KEY (`IDJogo`) REFERENCES `jogo` (`IDJogo`) ON
  DELETE CASCADE ON UPDATE CASCADE;

--
-- Limitadores para a tabela `mensagens`
--
ALTER TABLE `mensagens`
  ADD CONSTRAINT `chaveestrangeira3` FOREIGN KEY (`IDJogo`) REFERENCES `jogo`
  (`IDJogo`) ON DELETE CASCADE ON UPDATE CASCADE;
```



```
--  
-- Limitadores para a tabela `ocupacaolabirinto`  
--  
ALTER TABLE `ocupacaolabirinto`  
  ADD CONSTRAINT `jogo_jogo` FOREIGN KEY (`IDJogo`) REFERENCES `jogo` (`IDJogo`)  
ON DELETE CASCADE ON UPDATE CASCADE;  
  
--  
-- Limitadores para a tabela `sound`  
--  
ALTER TABLE `sound`  
  ADD CONSTRAINT `chaveestrangeira2` FOREIGN KEY (`IdJogo`) REFERENCES `jogo`  
(`IDJogo`) ON DELETE CASCADE ON UPDATE CASCADE;  
COMMIT;
```

2.6.2. Códigos dos Triggers Implementados

1. Nome Trigger: `after_setupmaze_insert`

Descrição: Este trigger após a inserção na tabela SetupMaze. A sua principal função é inicializar as salas na tabela labirinto para o jogo que se encontra em estado *running*. Garantido assim que a tabela *ocupacaolabirinto* *exibe todas as salas que existem* no labirinto.

Código:

```
DELIMITER //
```

```
CREATE TRIGGER after_setupmaze_insert  
AFTER INSERT ON SetupMaze  
FOR EACH ROW  
BEGIN  
  DECLARE current_idjogo INT;  
  DECLARE sala_counter INT DEFAULT 1;  
  
  -- Busca o IDJogo do jogo "running"  
  SELECT IDJogo INTO current_idjogo  
  FROM Jogo  
  WHERE Estado = 'running'  
  LIMIT 1;
```



```
IF current_idjogo IS NOT NULL THEN
    -- Remove salas antigas do mesmo jogo (se necessário)
    DELETE FROM ocupacaolabirinto
    WHERE IDJogo = current_idjogo;

    -- Insere novas salas (1 até numberrooms)
    WHILE sala_counter <= NEW.numberrooms DO
        INSERT INTO ocupacaolabirinto (
            IDJogo,
            NumeroMarsamisOdd,
            NumeroMarsamisEven,
            Sala,
            Tentativas
        ) VALUES (
            current_idjogo,
            0,
            0,
            sala_counter,
            0
        );
        SET sala_counter = sala_counter + 1;
    END WHILE;
END IF;
END //
```

DELIMITER ;

2. Nome Trigger: after_setupmaze_insert_update

Descrição: Este trigger segue a mesma lógica que o anterior mas para a operação Update.

Código:

```
DELIMITER //
```

```
CREATE TRIGGER after_setupmaze_insert_update
AFTER UPDATE ON SetupMaze
FOR EACH ROW
BEGIN
    DECLARE current_idjogo INT;
```




```
DECLARE sala_counter INT DEFAULT 1;

-- Busca o IDJogo do jogo "running"
SELECT IDJogo INTO current_idjogo
FROM Jogo
WHERE Estado = 'running'
LIMIT 1;

IF current_idjogo IS NOT NULL THEN
    -- Remove salas antigas do mesmo jogo (se necessário)
    DELETE FROM ocupacaolabirinto
    WHERE IDJogo = current_idjogo;

    -- Insere novas salas (1 até numberrooms)
    WHILE sala_counter <= NEW.numberrooms DO
        INSERT INTO ocupacaolabirinto (
            IDJogo,
            NumeroMarsamisOdd,
            NumeroMarsamisEven,
            Sala,
            Tentativas
        ) VALUES (
            current_idjogo,
            0,
            0,
            sala_counter,
            0
        );
        SET sala_counter = sala_counter + 1;
    END WHILE;
END IF;
END //
```

DELIMITER ;

3. Nome Trigger: after_medicoespassagens_insert

Descrição: Este trigger é ativado após a inserção na tabela *medicoesdepassagem*. O objetivo deste trigger é atualizar corretamente o número de marsamis Odd e Even na tabela *ocupacaolabirinto*. Refletido assim os movimentos no labirinto.



Código:

```
DELIMITER //
```



```
CREATE TRIGGER after_medicoespassagens_insert
AFTER INSERT ON medicoespassagens
FOR EACH ROW
BEGIN
    -- Apenas considera inserção na SalaDestino se SalaOrigem = 0
    IF NEW.SalaOrigem = 0 AND NEW.SalaDestino != 0 THEN
        INSERT INTO ocupacaolabirinto (IDJogo, Sala, NumeroMarsamisOdd,
NumeroMarsamisEven)
            VALUES (NEW.IDJogo, NEW.SalaDestino,
                IF(NEW.Marsami % 2 = 1, 1, 0),
                IF(NEW.Marsami % 2 = 0, 1, 0)
            )
        ON DUPLICATE KEY UPDATE
            NumeroMarsamisOdd = NumeroMarsamisOdd + IF(NEW.Marsami % 2 = 1, 1, 0),
            NumeroMarsamisEven = NumeroMarsamisEven + IF(NEW.Marsami % 2 = 0, 1, 0);
    END IF;

    -- Atualiza a sala de destino e origem apenas se a SalaOrigem for diferente de 0
    e SalaDestino diferente de 0
    IF NEW.SalaOrigem != 0 AND NEW.SalaDestino != 0 THEN
        -- Incrementa na sala destino
        INSERT INTO ocupacaolabirinto (IDJogo, Sala, NumeroMarsamisOdd,
NumeroMarsamisEven)
            VALUES (NEW.IDJogo, NEW.SalaDestino,
                IF(NEW.Marsami % 2 = 1, 1, 0),
                IF(NEW.Marsami % 2 = 0, 1, 0)
            )
        ON DUPLICATE KEY UPDATE
            NumeroMarsamisOdd = NumeroMarsamisOdd + IF(NEW.Marsami % 2 = 1, 1, 0),
            NumeroMarsamisEven = NumeroMarsamisEven + IF(NEW.Marsami % 2 = 0, 1, 0);

        -- Decrementa na sala origem
        UPDATE ocupacaolabirinto
        SET NumeroMarsamisOdd = NumeroMarsamisOdd - IF(NEW.Marsami % 2 = 1, 1, 0),
            NumeroMarsamisEven = NumeroMarsamisEven - IF(NEW.Marsami % 2 = 0, 1, 0)
        WHERE IDJogo = NEW.IDJogo AND Sala = NEW.SalaOrigem;
    END IF;
```



```
END //
```

```
DELIMITER ;
```

4. Nome Trigger: before_insert_mensagens_spam_check

Descrição: Este trigger é ativado antes da inserção na tabela *mensagens*. O objetivo deste trigger é se antes de ser inserida uma mensagem verifica se o tempo da última mensagem inserida com a mensagem a ser atualmente inserida é maior que o spamTol da tabela Jogo, de forma a evitar o SPAM de mensagens

Código:

```
DELIMITER //
```

```
CREATE TRIGGER before_insert_mensagens_spam_check
```

```
BEFORE INSERT ON mensagens
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE ultima_hora_escrita TIMESTAMP;
```

```
    DECLARE ultimo_sensor INT;
```

```
    DECLARE spam_tol INT;
```

```
    DECLARE diferenca_segundos INT;
```

```
    SET ultima_hora_escrita = NULL;
```

```
    SET ultimo_sensor = NULL;
```

```
    SET spam_tol = NULL;
```

```
    SELECT HoraEscrita, Sensor INTO ultima_hora_escrita, ultimo_sensor
```

```
    FROM mensagens
```

```
    WHERE HoraEscrita IS NOT NULL
```

```
    ORDER BY HoraEscrita DESC
```

```
    LIMIT 1;
```

```
    SELECT spamTol INTO spam_tol
```

```
    FROM jogo
```

```
    WHERE IDJogo = NEW.IdJogo
```



```
LIMIT 1;

IF spam_tol IS NULL OR spam_tol <= 0 OR ultima_hora_escrita IS NULL THEN
    SET diferenca_segundos = NULL;
ELSE
    SET diferenca_segundos = TIMESTAMPDIFF(SECOND, ultima_hora_escrita,
NEW.HoraEscrita);
END IF;

IF diferenca_segundos IS NOT NULL AND diferenca_segundos < spam_tol AND
NEW.Sensor = ultimo_sensor THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Inserção descartada: Tempo entre mensagens menor que
spamTol e Sensor igual ao último registro.';
END IF;
END //

DELIMITER ;
```

2.6.3. Códigos dos SPs Implementados

1. Nome SP: CreateDBUser

Descrição: Cria um novo utilizador da base de dados MySQL com username e password fornecidos. Este utilizador poderá depois ser associado a um perfil de utilizador na aplicação.

Código:

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `CreateDbUser`(IN `p_username`
VARCHAR(100), IN `p_password` VARCHAR(100))
BEGIN

IF p_username IS NULL OR CHAR_LENGTH(TRIM(p_username)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Não foi inserido nenhum username";
END IF;

IF p_password IS NULL OR CHAR_LENGTH(TRIM(p_password)) = 0 THEN
```



```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Não foi inserida nenhuma
password";
END IF;

SET @sql := CONCAT('CREATE USER \'', p_username, '\'@\'localhost\' IDENTIFIED BY
\'', p_password, '\');
PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;

SET @grant_sql := CONCAT('GRANT ALL PRIVILEGES ON pisd_sql.* TO \'',
p_username, '\'@\'localhost\');
PREPARE grant_stmt FROM @grant_sql;
EXECUTE grant_stmt;
DEALLOCATE PREPARE grant_stmt;

END$$
DELIMITER ;
```

2. Nome SP: Criar_utilizador

Descrição: Adiciona um novo utilizador à tabela utilizador, após validar o formato do e-mail e garantir que este não existe na base de dados. Também chama **CreateDbUser** para criar o utilizador MySQL correspondente.

Código:

```
DELIMITER $$
CREATE DEFINER='root'@'localhost' PROCEDURE `Criar_utilizador`(IN `p_Nome`
VARCHAR(100), IN `p_Telemovei` VARCHAR(12), IN `p_Tipo` VARCHAR(20), IN `p_Grupo`
VARCHAR(20), IN `p_Email` VARCHAR(50), IN `p_Password` VARCHAR(100))
BEGIN

DECLARE v_alreadyExists BOOLEAN DEFAULT FALSE;
DECLARE v_validEmail BOOLEAN DEFAULT FALSE;

IF p_Email IS NULL OR CHAR_LENGTH(TRIM(p_Email)) = 0 THEN SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = "Não foi inserido nenhum Email";END IF;
```



```
IF p_Password IS NULL OR CHAR_LENGTH(TRIM(p_Password)) = 0 THEN SIGNAL SQLSTATE
'45000' SET MESSAGE_TEXT = "Não foi inserida nenhuma Password";END IF;

CALL ValidEmail(p_Email,v_validEmail);

IF v_validEmail = FALSE THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Email com
formato Invalido";END IF;

CALL ExistsUtilizador(p_Email, v_alreadyExists);

IF v_alreadyExists = TRUE THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Este
utilizador já existe";END IF;

CALL CreateDbUser(p_Email,p_Password);
INSERT INTO utilizador (nome, telemovel, tipo, grupo, email)
VALUES (p_Nome, p_Telemovel, p_Tipo, p_Grupo, p_Email);
END$$
DELIMITER ;
```

3. Nome SP: Alterar_jogo

Descrição: Atualiza os dados de um jogo existentes na base de dados com base no seu ID. Permite modificar atributos como a descrição ou o spamTol, podendo ser feita apenas a alteração de um dos atributos ou dos dois ao mesmo tempo.

Código:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `Alterar_jogo`(IN `p_IDJogo` INT, IN
`p_JSONData` JSON)
BEGIN

DECLARE v_existsGame BOOLEAN;
DECLARE v_Tipo VARCHAR(3);
DECLARE v_CriadorJogo VARCHAR(100);
DECLARE v_EstadoJogo VARCHAR(20);
DECLARE v_User VARCHAR(100);
SET v_User := SUBSTRING_INDEX(SESSION_USER(), '@localhost', 1);
```



```
SELECT Tipo INTO v_Tipo FROM utilizador WHERE Email = v_User;
SELECT Jogador INTO v_CriadorJogo FROM jogo WHERE IDJogo = p_IDJogo;

CALL ExistsGame(p_IDJogo, v_existsGame);

IF v_existsGame = FALSE THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Jogo Inserido não existe";
END IF;

IF v_CriadorJogo <> v_User AND v_Tipo <> 'ADM' THEN SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = "O jogador não tem permissões para alterar este jogo";END IF;

SELECT Estado into v_EstadoJogo FROM jogo where IDJogo = p_IDJogo;

IF v_EstadoJogo = 'running' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
"Apenas pode alterar jogos no estado pending ou finished"; END IF;

UPDATE jogo
SET

Descricao =
COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_JSONData, '$.Descricao')),Descricao),
spamTol = COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_JSONData, '$.spamTol')),spamTol)
WHERE IDJogo = p_IDJogo;

END$$
DELIMITER ;
```

4. Nome SP: Criar_jogo

Descrição: Insere um novo registo na tabela de jogos com as informações fornecidas. Utilizando um input no formato JSON para uma melhor modularidade de informação fornecida.

Código:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `Criar_jogo`(IN `p_JSONData` JSON)
BEGIN
```



```
DECLARE v_User VARCHAR(100);
DECLARE v_Descricao TEXT;
DECLARE v_spamTol INT;

SET v_User := SUBSTRING_INDEX(SESSION_USER(), '@localhost', 1);

SET v_Descricao =
COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_JSONData, '$.Descricao')), "");
SET v_spamTol = COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_JSONData, '$.spamTol')), 2);

IF v_User IS NULL OR CHAR_LENGTH(TRIM(v_User)) = 0 THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT="Nenhum utilizador encontrado";END IF;
    IF CHAR_LENGTH(v_descricao) > 999 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT= "Descrição demasiado longa";
    END IF;

    IF v_spamTol < 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Valor não
pode ser nulo ou menor que 0";END IF;

INSERT INTO jogo(descricao, jogador, DataHorainicio, estado,spamTol)
VALUES (v_Descricao,v_User, NOW(), 'pending',v_spamTol);

END$$
DELIMITER ;
```

5. Nome SP: Eliminar_Jogo

Descrição: Remove um jogo da base de dados com base no respetivo identificador (ID). As restrições de integridade referencial, definidas com ações ON DELETE CASCADE, asseguram que todos os dados dependentes – como pontuações, jogadas ou associações a utilizadores – são automaticamente eliminados ou corretamente atualizados, garantindo a consistência dos dados.

Código:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `Eliminar_Jogo`(IN `p_IDJogo` INT)
BEGIN
```




```
DECLARE v_existsGame BOOLEAN;
DECLARE v_EstadoJogo VARCHAR(20);
DECLARE v_CriadorJogo VARCHAR(100);
DECLARE v_Tipo VARCHAR(3);
DECLARE v_User VARCHAR(100);
SET v_User := SUBSTRING_INDEX(SESSION_USER(), '@localhost', 1);
SELECT Tipo INTO v_Tipo FROM utilizador WHERE Email = v_User;
SELECT Jogador INTO v_CriadorJogo FROM jogo WHERE IDJogo = p_IDJogo;
IF (p_IDJogo IS NULL) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Não foi
inserido nenhum ID de Jogo";END IF;

CALL ExistsGame(p_IDJogo,v_existsGame);

IF(v_existsGame = FALSE) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Não
existe nenhum jogo com esse ID";END IF;

IF v_CriadorJogo <> v_User AND v_Tipo <> 'ADM' THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT = "O jogador não tem permissões para alterar este jogo";END IF;

SELECT Estado into v_EstadoJogo FROM jogo where IDJogo = p_IDJogo;

IF v_EstadoJogo = 'running' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Não
pode eliminar jogos que estão a decorrer"; END IF;

DELETE FROM jogo WHERE IDJogo=p_IDJogo;
END$$
DELIMITER ;
```

6. Nome SP: ExistsGame

Descrição: Verifica se existe um jogo na base de dados com um determinado ID retornando um resultado boolean.

Código:

```
DELIMITER $$
```



```
CREATE DEFINER=`root`@`localhost` PROCEDURE `ExistsGame`(IN `p_IDJogo` INT, OUT  
`p_exists` BOOLEAN)  
BEGIN  
DECLARE count INT;  
  
IF p_IDJogo IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Não foi  
inserido nenhum jogo";END IF;  
SELECT COUNT(*) INTO count from jogo WHERE jogo.IDJogo = p_IDJogo;  
  
SET p_exists = (count > 0);  
END$$  
DELIMITER ;
```

7. Nome SP: ExistsUtilizador

Descrição: Verifica se existe um utilizador na base de dados com um determinado e-mail. Retorna um valor de controlo que permite validar a existência do utilizador.

Código:

```
DELIMITER $$  
CREATE DEFINER=`root`@`localhost` PROCEDURE `ExistsUtilizador`(IN `p_Email`  
VARCHAR(50), OUT `p_exists` BOOLEAN)  
BEGIN  
DECLARE count INT;  
DECLARE v_validEmail BOOLEAN DEFAULT FALSE;  
CALL ValidEmail(p_Email,v_validEmail);  
  
IF p_Email IS NULL OR CHAR_LENGTH(TRIM(p_Email)) = 0 THEN SIGNAL SQLSTATE '45000'  
SET MESSAGE_TEXT = "Não foi inserido nenhum utilizador";END IF;  
IF v_validEmail = FALSE THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Email com  
formato invalido";END IF;  
SELECT COUNT(*) INTO count FROM utilizador  
WHERE utilizador.Email = p_Email;  
SET p_exists = (count > 0);  
  
END$$
```



```
DELIMITER ;
```

8. Nome SP: GetGames

Descrição: Retorna a lista de jogos existentes na base de dados de um determinado grupo.

Código:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `GetGames`(IN `p_grupo` INT)
BEGIN

SELECT * FROM jogo j INNER JOIN utilizador u ON j.jogador = u.Email WHERE u.Grupo
= p_grupo;

END$$
DELIMITER ;
```

9. Nome SP: Remover_Utilizador

Descrição: Remove um utilizador da base de dados, depois de confirmar que o utilizador existe. Apenas pode ser executada por um administrador.

Código:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `Remover_utilizador`(IN `p_utilizador`
VARCHAR(50))
BEGIN
DECLARE v_existsUtilizador BOOLEAN;

IF (p_utilizador IS NULL OR p_utilizador = '') THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT = "Não foi inserido nenhum utilizador";END IF;

CALL ExistsUtilizador(p_utilizador,v_existsUtilizador);
```



```
IF (v_existsUtilizador = FALSE) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
"Utilizador não existe";END IF;

DELETE FROM utilizador WHERE Email=p_utilizador;
SET @drop_sql := CONCAT('DROP USER \', p_utilizador, '\'@\localhost\');
PREPARE stmt FROM @drop_sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
END$$
DELIMITER ;
```

10. Nome SP: Correr_Jogo

Descrição: Altera o estado de um jogo para 'running', depois de verificar se o jogo existe, se ainda não começou e se o utilizador que o está a tentar iniciar é o criador do jogo ou um administrador.

Código:

```
DELIMITER $$
CREATE DEFINER=`root`@\localhost` PROCEDURE `Correr_Jogo`(IN `p_IDJogo` INT)
BEGIN

DECLARE v_CriadorJogo VARCHAR(100);
DECLARE v_Tipo VARCHAR(3);
DECLARE v_ExisteJogo BOOLEAN DEFAULT FALSE;
DECLARE v_EstadoJogo VARCHAR(20);
DECLARE v_User VARCHAR(100);
IF p_IDJogo IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Nenhum jogo
foi inserido";END IF;

SET v_User := SUBSTRING_INDEX(SESSION_USER(), '@localhost', 1);
SELECT Tipo into v_Tipo FROM utilizador WHERE Email = v_User;
SELECT Jogador INTO v_CriadorJogo FROM jogo WHERE IDJogo = p_IDJogo;
IF v_User <> v_CriadorJogo AND v_Tipo <> 'ADM' THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT = "Não tem permissões para alterar este jogo";END IF;

CALL ExistsGame(p_IDJogo, v_ExisteJogo);
```



```
IF v_ExisteJogo = FALSE THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "O jogo
inserido não existe";END IF;

SELECT Estado into v_EstadoJogo FROM jogo where IDJogo = p_IDJogo;

IF v_EstadoJogo = 'running' OR v_EstadoJogo = 'finished' THEN SIGNAL SQLSTATE
'45000' SET MESSAGE_TEXT = "Apenas pode começar jogos no estado pending"; END IF;

UPDATE jogo SET Estado = 'running' WHERE IDJogo = p_IDJogo;

END$$
DELIMITER ;
```

11. Nome SP: ValidEmail

Descrição: Para validar o email (por exemplo se contém é do formato
xxxxxx@xxxx.xx)

Código:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `ValidEmail`(IN `p_Email` VARCHAR(50) ,
OUT `p_isValid` BOOLEAN)
BEGIN

IF p_Email REGEXP '^[A-Za-z0-9_.%+-]+@[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+' THEN SET
p_isValid =TRUE; ELSE
SET p_isValid = FALSE;
END IF;

END$$
DELIMITER ;
```

12. Nome SP: Alterar_utilizador



Descrição: Stored Procedure que permite alterar a informação de um utilizador com recurso a JSON.

Código:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `Alterar_utilizador`(IN `p_email`
VARCHAR(50), IN `p_json_data` JSON)
BEGIN
    UPDATE utilizador
    SET
        Nome = COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_json_data, '$.Nome')), Nome),
        Telemovel = COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_json_data,
        '$.Telemovel')), Telemovel),
        Tipo = COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_json_data, '$.Tipo')), Tipo),
        Grupo = COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_json_data, '$.Grupo')),
        Grupo),
        Email = COALESCE(JSON_UNQUOTE(JSON_EXTRACT(p_json_data, '$.Email')),
        Email)
    WHERE Email = p_email;
END$$
DELIMITER ;
```